

SAPIENZA UNIVERSITÀ DI ROMA

Tecniche di Programmazione Funzionale e Imperativa

*libro del corso:
slide del professor Salvo*

STOP DOING FUNCTIONAL PROGRAMMING

- **FOR LOOPS** WERE NOT SUPPOSED TO BE GIVEN NAMES
- YEARS OF **HOFs** yet NO REAL-WORLD USE FOUND for going higher than **CALLBACKS**
- Wanted to go higher anyway for a laugh? We had a tool for that: It was called **AbstractWidgetLocalizerManagerFactoryBe**
- "Yes please **FOLD** over this collection . Please give me a **CURRIED** function" - Statements dreamed up by the utterly Deranged

LOOK at what **Functional Programmers** have been demanding your Respect for all this time, with all the boilerplate & compilers we built for them **(These are REAL Functions, done by REAL Functional Programmers):**

```
set :: Lens' s a -> a -> s -> s
set in v s = runIdentity (in (const (Identity v)) s)

over :: Lens' s a -> (a -> a) -> s -> s
over in g s = runIdentity (in (Identity . g) s)

view :: Lens' s a -> s -> a
view in v s = getConst (in Const s)
```

```
class Prefunction p where
  clamp :: (a -> a) -> (b -> b) -> p a b -> p a' b'

  clamp :: (a' -> a) -> p a b -> p a' b
  clamp f = clamp f id

  rmap :: (b -> b') -> p a b -> p a b'
  rmap f = clamp id f
```

```
(>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
(>>) :: Maybe a -> Maybe b -> Maybe b
return :: a -> Maybe a
```

????? ?????? ??????????????????

"Hello I would like a Monad m please"

They have played us for absolute fools

aglaia norza

1 giugno 2026

✉ thisisaglaia@gmail.com

🐙 github.com/AglaiaNorza

1	Le Basi	4
1.1	Concetti fondamentali	4
1.2	Sintassi, Funzioni e Pattern Matching	5
1.2.1	Definizione e Applicazione	5
1.2.2	Lambda Espressioni e Combinatori	5
1.2.3	Pattern matching	5
1.2.4	Scope locale: <code>let</code> e <code>where</code>	6
1.3	Sistema dei Tipi e Classi	8
1.3.1	Type Signatures	8
1.3.2	Type Classes	8
1.3.3	Definizione di Tipi: <code>type</code> e <code>data</code>	9
1.3.4	Il costruttore di tipo <code>Maybe</code>	9
1.3.5	Istanze e Derivazione (<code>deriving</code>)	10
1.3.6	Tipabilità e Let-Polymorphism	10
1.3.7	Semantica Denotazionale: ricorsione e punti fissi	11
1.4	Strutture Dati Predefinite	12
1.4.1	Tipi di Base e Tuple	12
1.4.2	Liste	12
1.5	Funzion(al)i di Ordine Superiore	14
1.5.1	Famiglia dei <code>fold</code> (schemi di ricorsione)	14
1.6	Alcuni esempi di stile funzionale	15
2	Lambda Calcolo	17
2.1	Sintassi e Computazione	17

2.2	Variabili e Combinatori	17
2.2.1	Combinatori noti e strutture di controllo	18
2.3	Ricorsione e Combinatori di Punto Fisso	18
3	Temi avanzati di programmazione funzionale	20
3.1	Fusion law per foldr	20
3.2	Lazy evaluation	22
3.2.1	Weak Head Normal Form (WHNF)	22
3.2.2	Call-by-name (CBN)	24
3.2.3	Sharing	24
3.3	Funzioni non strette	25
3.4	Tecniche di ottimizzazione	28
3.4.1	Impatto della Laziness sulla Complessità Asintotica	28
3.4.2	Parametri Accumulatori	29
3.4.3	Tupling	30
3.4.4	Strictness e gestione dello spazio	31
3.4.5	Zucchero Sintattico: <code>\$</code> e <code>\$!</code>	33
3.4.6	<code>scanl</code> e <code>scanr</code>	33
3.5	Strutture dati infinite	34
3.5.1	Definizioni naif e circolari	34
3.6	Liste infinite come limiti	36
3.6.1	Domini, ordine e limiti	36
3.7	Teoremi di punto fisso	39
3.8	Induzione e co-induzione	40
3.8.1	Insiemi induttivi (e relativi funzionali)	41
3.8.2	Insiemi coinduttivi (e relativi funzionali)	42
3.8.3	Bisimulazione	44
3.8.4	Dimostrazione per approssimazione	44
3.9	Funtori e Applicativi	45
3.9.1	Funtori	46
3.9.2	Applicativi	50
3.10	Monadi Base	55
3.10.1	Introduzione	55
3.10.2	Monadi (shall I be pure or impure?)	56
3.10.3	Alcune monadi note	57
3.10.4	Monade State Transformer	58

3.10.5 Do notation	59
3.10.6 Monad Laws	62
3.11 Input/Output	62
3.11.1 Azioni base	63
3.11.2 Strictness dell'I/O	64
3.12 State Thread Monad	65
3.13 Kleisli composition (composizione monadica)	67
3.14 Functor e Applicative come Monad	69
3.14.1 Definire <code>fmap</code> e <code><*></code>	69
3.15 Funzione <code>join</code>	70
3.15.1 Definizione di bind tramite join	70
3.16 Generic Programming	71
3.16.1 <code>mapM</code> e <code>filterM</code>	71
3.17 Monoidi, Foldable e Traversable	72
3.17.1 <code>Monoid</code>	72
3.17.2 <code>Foldable</code>	73
3.17.3 <code>Traversable</code>	74

1.1 Concetti fondamentali

- **Valutazione lazy:** Haskell valuta le espressioni solo quando è strettamente necessario (permette di gestire agevolmente strutture dati infinite)
- **Trasparenza referenziale:** non essendoci assegnazioni o memoria mutabile (assenza di side-effects), una variabile identifica sempre lo stesso oggetto (permette di manipolare algebricamente i programmi)
- **Polimorfismo e type inference:** molte funzioni operano su tipi generici (es. `a`, `b`). Il compilatore inferisce automaticamente il *tipo principale* (il più generale possibile); tutti gli altri tipi corretti sono sue istanze.
- **Non-terminazione, Bottom e undefined:** in Haskell, ogni tipo possiede implicitamente un valore speciale (chiamato *bottom*, indicato matematicamente con $\perp$) che rappresenta un'eccezione fatale o una computazione che entra in un loop infinito
 - **la costante undefined:** è la materializzazione a livello di codice del valore $\perp$ - poiché una computazione che fallisce o non termina può avvenire in qualsiasi contesto (mentre si aspetta un numero, un booleano, ecc.), `undefined` possiede il tipo più generale e polimorfo possibile: `a` (ovvero $\forall \alpha. \alpha$)
 - **l'occurs check:** nel lambda calcolo puro, il loop infinito per eccellenza è il combinatore $\Omega = (\lambda x.xx)(\lambda x.xx)$. In Haskell, l'auto-applicazione `$\lambda x.xx$` **non è tipabile**. Perché x possa prendere se stesso come argomento, il suo tipo α dovrebbe essere uguale a una funzione che prende α come parametro e restituisce β ($\alpha = \alpha \rightarrow \beta$). Questo creerebbe un tipo infinito (durante l'inferenza dei tipi, il compilatore esegue un controllo chiamato *occurs check* che impedisce alla variabile di tipo α di apparire all'interno della sua stessa definizione, rifiutando di conseguenza l'espressione)
 - **ricorsione esplicita:** poiché il type system ci vieta di creare loop tramite l'auto-applicazione anonima di `$\lambda x.xx$` , in Haskell la non-terminazione si esprime unicamente attraverso la ricorsione esplicita. Definendo ad esempio `omega' = omega'`, creiamo intenzionalmente una computazione divergente che il compilatore accetta, assegnandole il tipo più generale `a`.
- **Ideologia funzionale:** l'ideologia funzionale favorisce l'assemblaggio di funzioni semplici tramite composizione (`.`). Si pensa in termini di trasformazioni globali di strutture dati, per poi raffinare in un secondo momento i colli di bottiglia prestazionali.

1.2 Sintassi, Funzioni e Pattern Matching

1.2.1 Definizione e Applicazione

- l'applicazione di funzione: **associa a sinistra** e si scrive senza virgole (`f x y`).
- **operatori infissi vs prefissi**: le funzioni binarie si usano in modo infisso tra backtick (`10 `div` 2`), gli operatori infissi in modo prefisso tra parentesi (`(+) 3 4`).

1.2.2 Lambda Espressioni e Combinatori

Le funzioni anonime si scrivono con backslash (`\` simula λ).

```

1  -- identita' (combinatore I)
2  i :: t -> t
3  i x = x           -- definizione standard
4  i' = \x -> x      -- lambda espressione
5
6  -- combinatore K (proiettore del primo argomento) (primo assioma di Hilbert)
7  k :: t1 -> t2 -> t1
8  k' = \x y -> x
9
10 -- combinatore S (sostituzione) (secondo assioma di Hilbert)
11 s :: (t2 -> t1 -> t) -> (t2 -> t1) -> t2 -> t
12 s x y z = x z (y z)

```

1.2.3 Pattern matching

Il pattern matching decompone le strutture dati in base alla loro forma sintattica. L'ordine delle clausole è strettamente sequenziale (viene scelta la prima dall'alto che fa matching).

- **mancanza di unificazione**: non si possono usare variabili identiche nel pattern per verificarne l'uguaglianza (es. `f x x = ...` è invalido)
- **underscore (`_`)**: variabile anonima ("don't care") per valori non rilevanti
- **as-pattern (`@`)**: permette di dare un nome all'intera struttura mentre la si decompone (es. `xs@(_:txs)`)
- **guardie (`|`)**: permettono di definire il comportamento di una funzione valutando sequenzialmente una serie di *predicati booleani* (condizioni logiche); a differenza del pattern matching che controlla la *forma* sintattica dei dati, le guardie valutano *valori e relazioni* (es. `x > 0` , `x == y`)
 - l'ultima guardia è tipicamente `otherwise` (che nel Prelude di Haskell è semplicemente un alias per `True`), che funge da caso di default ("catch-all") per assicurarsi che la funzione restituisca sempre un valore.

```

1  confronta x y
2  | x == y    = "i numeri sono uguali" -- supera la mancanza di unificazione
3  | x > y     = "il primo e' maggiore"
4  | otherwise = "il secondo e' maggiore"

```

- **equazioni multiple (definizione per casi)**: il modo più comune di fare pattern matching è definire la funzione scrivendo più "intestazioni" separate; il compilatore le prova dall'alto verso

il basso: se gli argomenti combaciano con la forma (pattern) della riga, esegue quell'equazione, altrimenti controlla la successiva.

```
1 myAnd True True = True
2 myAnd _ _ = False
```

– è meno comune, ma è possibile fare pattern-matching anche nelle *lambda-espressioni*:

```
1 myFst' = \ (x, y) -> x
```

1.2.4 Scope locale: **let** e **where**

- **let ... in**: costrutto standalone per definire variabili/funzioni locali (utile per strutturare/destrutturare tuple nelle ricorsioni)
- **where**: clausola che qualifica la parte destra di una definizione appena conclusa

```
1 fibAux n = let (f, fPrec) = fibAux (n-1) in (f + fPrec, f)
2 -- equivalente a:
3 fibAux' n = (f + fPrec, f) where (f, fPrec) = fibAux' (n-1)
```

differenza tra **let** e **where**

- **let ... in** è un'espressione: produce un valore, e si può utilizzare ovunque sia legale inserire un'espressione (in una lista, in un'operazione, come argomento di funzione ecc.)

```
1 4 + (let x = 2 in x * 3) -- risultato: 10
```

- **where** è una clausola: non produce un valore - serve a dichiarare "a valle" qualcosa che si è usato "a monte"
- **let** ha un approccio bottom-up: prima si definiscono le variabili e poi si utilizzano
- **where** ha un approccio top-down: prima si scrive la logica della funzione, e poi si spiega cosa significano i vari pezzi (è spesso più leggibile per funzioni lunghe)

I due costrutti sono molto diversi anche per il loro comportamento con le **guardie** (**|**):

- le variabili definite in un **where** sono visibili a tutte le guardie

```
1 -- condivisione tra guardie
2 analizzaRaggio r
3 | area > 100 = "cerchio grande"
4 | area < 10 = "cerchio piccolo"
5 | otherwise = "cerchio medio"
6 where area = pi * r^2 -- 'area' calcolata una volta e visibile a tutti i pipe
```

- un **let** si può usare in una guardia solo dopo il simbolo **=**, ovvero nel corpo di una specifica guardia - la variabile è quindi invisibile alle altre guardie

```
1  -- un codice di questo tipo non funziona:  
2  analizzaRaggio r  
3    | area > 100 = "cerchio grande" -- errore! 'area' non esiste ancora  
4    | otherwise = let area = pi * r^2 in "cerchio medio" -- 'area' esiste  
                   solo qui dentro
```


1.3 Sistema dei Tipi e Classi

Haskell è **statically type-safe**: se un programma è tipato correttamente, nessun errore di tipo si verifica durante l'esecuzione.

1.3.1 Type Signatures

In Haskell, grazie al meccanismo di type inference, il compilatore è in grado di dedurre automaticamente il tipo più generale per quasi ogni espressione. Tuttavia, è considerata un'ottima pratica (e in alcuni casi più complessi è strettamente necessario) dichiarare esplicitamente il tipo di una funzione subito sopra la sua definizione.

Questo si fa usando l'operatore `::` (che si legge “ha tipo”). Dichiarare esplicitamente i tipi è utile ai fini della leggibilità del codice e permette di “forzare” il tipo desiderato, aiutando il compilatore a individuare eventuali errori logici in modo molto più preciso.

```
1  -- dichiarazione esplicita del tipo: prende due Int e restituisce un Int
2  somma :: Int -> Int -> Int
3  somma x y = x + y
```

1.3.2 Type Classes

Le classi limitano il polimorfismo imponendo vincoli (o “interfacce”) sui tipi. Più che semplici insiemi, rappresentano classificazioni di tipo algebrico: raggruppano i tipi su cui sono definite certe operazioni. Le classi possono formare gerarchie, dove una sottoclasse “estende” la superclasse ereditandone le operazioni e aggiungendone di nuove.

- **Eq (Equality Types)**: comprende i tipi che ammettono l'uguaglianza (`==`) e la sua negazione (`/=`). Tutti i tipi base (`Bool`, `Int`, `Float`, ecc.) sono istanze di `Eq`, così come le liste e tuple costruite su tipi `Eq`

definizione interna di `Eq`:

```
1  class Eq a where
2    (==), (/=) :: a -> a -> Bool
3    -- le classi possono contenere codice (metodi di default)
4    x /= y = not (x == y)
```

- **Ord (Ordered Types)**: sottoclasse di `Eq` - tipi ordinabili che supportano operazioni come `<=`, `<`, `>=`, `>`, `min` e `max`.
 - per definire un'istanza di `Ord`, è sufficiente fornire il codice per l'operatore `<`, il resto può essere interdefinito.
- **Num**: la classe più generica per i tipi numerici. È sottoclasse di `Eq` e `Show`. Implementa operazioni aritmetiche base (`+`, `-`, `*`), oltre a valore assoluto (`abs`) e segno (`signum`).

i numeri interi costanti sono polimorfi rispetto alla classe `Num`:

```
1  > :t 3
2  3 :: Num a => a  -- 3 puo' essere un Int, Integer, Float, ma non un Char
```

- **Altre classi utili:**

- **Show**: tipi che possono essere stampati (convertiti in `String`) tramite il metodo `show`. Se provate a stampare a video il risultato di una funzione che non appartiene a `Show` (es. una lambda anonima come `\x -> x`), l'interprete darà errore.
- **Read**: permette di trasformare una `String` in un valore tramite il metodo `read`
- **Integral**: sottoclasse di `Num` per i numeri interi (es. `Int`, `Integer`); aggiunge le operazioni di divisione intera `div` e modulo `mod`
- **Fractional**: sottoclasse di `Num` per i numeri frazionari (es. `Float`, `Rational`); aggiunge l'operatore di divisione reale `(/)` e `recip`

1.3.2.1 Vincoli di classe

I vincoli di classe si indicano prima del tipo vero e proprio, separati dalla doppia freccia `=>`. Vincoli multipli si racchiudono tra parentesi e indicano che la funzione opera su un tipo che deve soddisfare tutte le classi elencate.

```
1  -- esempio: mcd richiede che il tipo 'a' supporti sia l'ordine (<, ==)
2  -- sia le operazioni aritmetiche (-)
3  mcd :: (Ord a, Num a) => a -> a -> a
4  mcd x y
5    | x == y    = x
6    | x < y     = mcd (y - x) x
7    | otherwise = mcd (x - y) y
```

1.3.3 Definizione di Tipi: `type` e `data`

- **type (sinonimi)**: danno nuovi nomi a tipi esistenti (es. `type Casella = (Int, Int)`)
- **data (tipi algebrici)**: costruiscono nuovi tipi tramite costruttori finiti, parametrici o ricorsivi. Possono essere dichiarati istanze di classi.

```
1  data SetteNani = Eolo | Pisolo | Brontolo           -- finiti
2  data Either a b = Left a | Right b                 -- unioni disgiunte
3  data Maybe a = Just a | Nothing                    -- gestione fallimenti
4  data List a = Nil | Cons a (List a)                -- ricorsivi/induttivi
5  data BTree a = Foglia a | Radice (BTree a) (BTree a)
```

1.3.4 Il costruttore di tipo `Maybe`

Il tipo `Maybe` è il tipo di computazioni che possono fallire. In Haskell, è il tipo con cui si rappresentano comunemente le eccezioni in maniera sicura e controllata dal sistema di tipi (corrisponde a `optional`)

Un valore di tipo `Maybe` può assumere due forme:

- `Just v`: contiene un valore `v` (la computazione ha avuto successo e restituisce qualcosa)
- `Nothing`: non contiene nulla (rappresenta una computazione indefinita o fallita)

Si può utilizzare per le *funzioni parziali* (che non sono definite per tutti i possibili valori di input (eg divisione per zero)). Tramite il pattern matching, possiamo “spacchettare” il dato per estrarne il contenuto.

```

1  -- definizione del tipo predefinito
2  data Maybe a = Just a | Nothing
3
4  -- 1. costruzione di un Maybe: divisione sicura
5  -- Se il divisore e' 0 restituisce Nothing, altrimenti restituisce Just il
   risultato
6  safediv :: Integral a => a -> a -> Maybe a
7  safediv n 0 = Nothing
8  safediv n m = Just (n `div` m)
9
10 -- 2. destrutturare un Maybe: funzione "inversa" per estrarre il valore
11 extract :: Maybe a -> a
12 extract (Just n) = n
13 extract Nothing = undefined
14
15 -- extract (safediv 30 0) -> *** Exception: Prelude.undefined

```

1.3.5 Istanze e Derivazione (deriving)

La keyword `deriving` è un meccanismo automatico che permette al compilatore di generare le istanze per le classi predefinite senza dover scrivere codice manuale.

- **Eq**: genera un'uguaglianza basata sulla sintassi
- **Ord**: deriva l'ordine lessicografico in base alla sequenza dei costruttori
- **Show/Read**: la conversione usa i nomi dei costruttori come stringhe
- **Enum**: permette di generare liste per enumerazione (es. `[Lun..Ven]`)

```

1  -- istanze automatiche tramite deriving
2  data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
3              deriving (Eq, Ord, Show, Enum, Read)
4
5  -- esempio di istanza manuale
6  instance Eq Bool where
7      False == False = True
8      True  == True   = True
9      _    == _      = False

```

1.3.6 Tipabilità e Let-Polymorphism

- Non tutto è tipabile in Haskell. Il classico esempio è l'auto-applicazione pura:
 - `omega = \x -> x x` non è tipabile perché richiederebbe un tipo infinito ($t = t \rightarrow t_1$), portando a una teoria dei tipi non decidibile (e quindi a tipi non inferibili dal compilatore)
- Dal punto di vista del calcolo, `let x = N in M` è equivalente all'applicazione di una lambda: `(\x -> M) N`. Tuttavia, il compilatore Haskell tratta i due casi in modo diverso durante l'inferenza dei tipi.
- **Let-Polymorphism**:

(per più informazioni, vedi anche [appunti del corso "Linguaggi di programmazione"](#))

Il costrutto `let` permette di assegnare a una variabile un tipo polimorfo che viene "generalizzato" prima di essere usato nel corpo (`in`).

- **tipabile:** `let x = \y -> y in x x` è tipabile.
Haskell vede che `x` è l'identità ($a \rightarrow a$) e, nel corpo `x x`, può istanziare il primo `x` come $(a \rightarrow a) \rightarrow (a \rightarrow a)$ e il secondo come $(a \rightarrow a)$.
- **non tipabile:** `(\x -> x x) (\y -> y)` **non** è tipabile.
Qui il compilatore deve tipare la funzione `\x -> x x` prima di sapere che `x` sarà l'identità. Come visto sopra, `\x -> x x` è intrinsecamente non tipabile.

Lemma 1: **Terminazione**

Tutti i lambda-termini tipabili nel sistema dei tipi di Haskell (che non usino ricorsione esplicita tramite equazioni) **terminano**.

1.3.7 Semantica Denotazionale: ricorsione e punti fissi

Haskell è un linguaggio dichiarativo: una definizione non è un comando, ma un'equazione. Quando definiamo una funzione ricorsiva f , stiamo scrivendo un'equazione del tipo $f = \mathcal{E}(f)$, dove $\mathcal{E}$ è un'espressione che contiene f stessa.

- **Bottom** ($\perp$) rappresenta il “non-valore”. Indica un calcolo che non termina (loop infinito) o che fallisce (errore). In Haskell, ogni tipo t contiene implicitamente $\perp$.
- Se la definizione è un'equazione, la “semantica” (ovvero la funzione reale prodotta dal compilatore) deve essere una soluzione dell'equazione. Matematicamente, una soluzione di $f = T(f)$ è un **punto fisso** del funzionale T .

Approfondimento: ricorsione e non-terminazione: `omega'`

Prendiamo la funzione

```
1 omega' x = omega' x
2 -- > :t omega'
3 -- omega' :: t -> t1
```

Questa funzione ha il tipo più generico possibile ($t \rightarrow t1$) perché, non terminando mai, non produce mai un valore reale.

Rimuovendo x , otteniamo $\text{omega}' = I(\text{omega}')$, dove I è l'identità.

Qualsiasi funzione f soddisfa $f = I(f)$. Quale scegliere?

- Haskell sceglie la funzione che “fa il minimo indispensabile” per soddisfare l'equazione. Si usa l'ordinamento $\sqsubseteq$ dove $f \sqsubseteq g$ significa che g è “più definita” di f .

- **Costruzione di Kleene (least-fixpoint):**

il computer “costruisce” la funzione partendo dal nulla ($\perp$):

$$f_0 = \perp \quad (\text{non so nulla})$$

$$f_1 = T(f_0) \quad (\text{applico la definizione una volta})$$

$$f_\infty = \text{limite della catena} = \text{lfp}(T)$$

Nel caso di `omega'`, la catena è $\{\perp, \perp, \dots\}$. Il limite è $\perp$. La semantica di `omega'` è dunque la funzione che restituisce sempre “non-valore”.

Il punto fisso è quindi la “semantica” perché è l’unico modo per dare un valore univoco e calcolabile a una definizione circolare. Il *minimo* punto fisso garantisce che il significato sia quello “prodotto” effettivamente dal calcolo ricorsivo.

myundefined

Mentre `omega'` è una *funzione* che non termina, possiamo definire un *valore* che non termina:

```
1 myUndefined = myUndefined
2 -- > :t myUndefined
3 -- myUndefined :: a
```

- Poiché `myUndefined` ha tipo `a`, esso **appartiene a tutti i tipi**. È l’implementazione “home-made” della costante `undefined` di Haskell. Rappresenta il valore $\perp$ per ogni possibile tipo del linguaggio.
- Matematicamente, `myUndefined` è il **punto fisso dell’identità**:
 - (1) la definizione $x = x$ corrisponde all’equazione funzionale $x = I(x)$
 - (2) la soluzione di questa equazione è, per definizione, il punto fisso dell’identità I
 - (3) poiché Haskell usa la semantica del minimo punto fisso, e l’identità applicata al valore nullo ($\perp$) restituisce ancora $\perp$, il risultato è la non-terminazione
 - (4) questo spiega perché `myUndefined` ha tipo `a`: non essendo “sceso” verso nessun valore specifico (come un intero o una stringa), rimane una pura astrazione di errore/loop che può “fingere” di essere qualsiasi tipo.

Haskell permette quindi di manipolare la non-terminazione come un valore reale. Ogni tipo in Haskell è in realtà “puntato” (*pointed*), ovvero contiene sempre almeno un valore: $\perp$.

1.4 Strutture Dati Predefinite

1.4.1 Tipi di Base e Tuple

- **Booleani**: `Bool` (`True`, `False`). Operatori: `not`, `&&`, `||`.
- **Interi**: `Int` (dimensione fissa), `Integer` (precisione illimitata).
- **Tuple**: n-uple fisse e disomogenee (es. `(String, Bool)`)
offrono le funzioni *su coppie*: `fst`, `snd` (prendono primo e secondo argomento)

1.4.2 Liste

Sequenze di elementi omogenei. Definite dal costruttore vuoto `[]` e dall’operatore infisso `cons` `(:)`. Le stringhe sono `[Char]`.

Zucchero sintattico: `[1,2,3]` equivale a `1:2:3:[]`.

```
1 -- enumerazioni
2 [m..n]           -- range finito (lista con elementi da m a n)
3 [m, n..p]        -- range con step (es. [1, 3..11] dispari)
```

```
4 [m..]           -- lista infinita (lazy)
```

Haskell supporta le **list comprehension** (definizioni set-like del contenuto di una lista)

```
1 -- list comprehension (spesso tradotte internamente in map/filter)
2 [x^2 | x <- [1..5]]           -- quadrati
3 [x | x <- xs, x `mod` 2 == 0]   -- filtri / guardie
4 [(x,y) | x <- xs, y <- ys]     -- prodotto cartesiano
```

1.4.2.1 Funzioni su Liste

- **Estrazione:** `head`, `tail`, `last`, `init`, `(!!)` per estrarre all'indice n

```
1 head [1, 2, 3] -- risultato: 1
2 tail [1, 2, 3] -- risultato: [2, 3]
3 last [1, 2, 3] -- risultato: 3
4 init [1, 2, 3] -- risultato: [1, 2]
5 [1, 2, 3] !! 1 -- risultato: 2
```

- **Ispezione:** `null`, `length`

```
1 null []           -- risultato: True
2 length [1, 2, 3] -- risultato: 3
```

- **Taglio:** `take n`, `drop n`, `splitAt n` (tupla con `take` e `drop`).

```
1 take 2 [1, 2, 3, 4] -- risultato: [1, 2]
2 drop 2 [1, 2, 3, 4] -- risultato: [3, 4]
3 splitAt 2 [1, 2, 3, 4] -- risultato: ([1, 2], [3, 4])
```

- **Generali:** `++` (concatenazione), `reverse`

```
1 [1, 2] ++ [3, 4] -- risultato: [1, 2, 3, 4]
2 reverse [1, 2, 3] -- risultato: [3, 2, 1]
```

- **Matematica/logica:** `sum`, `minimum`, `maximum`, `and`, `or`, `any p`, `all p` (predicati su almeno uno o tutti)

```
1 sum [1, 2, 3]           -- risultato: 6
2 minimum [5, 2, 9]       -- risultato: 2
3 maximum [5, 2, 9]       -- risultato: 9
4 and [True, False]       -- risultato: False
5 or [True, False]        -- risultato: True
6 any (> 2) [1, 2, 3]     -- risultato: True (almeno uno è > 2)
7 all (> 0) [1, 2, 3]     -- risultato: True (tutti sono > 0)
```

- **Liste annidate/multiple:** `concat` (appiattisce liste di liste), `zip` (unisce liste in coppie, fermandosi alla più corta)

```

1 concat [[1, 2], [3, 4]] -- risultato: [1, 2, 3, 4]
2 zip [1, 2] ['a', 'b', 'c'] -- risultato: [(1, 'a'), (2, 'b')]

```

1.5 Funzion(al)i di Ordine Superiore

I funzionali sono funzioni che prendono altre funzioni come argomento o le restituiscono come risultato. Favoriscono la **composizionalità**, permettendo di costruire programmi complessi tramite piccoli blocchi riutilizzabili.

- **curryficazione**: si fonda sull'isomorfismo $A \times B \rightarrow C \cong A \rightarrow (B \rightarrow C)$. Le versioni curryficate permettono di passare un solo argomento alla volta, favorendo l'applicazione parziale.
- **η -regola**: è possibile omettere un parametro in una definizione se esso compare identico alla fine di entrambi i lati dell'equazione.

```

1 -- composizione (.) : Equivale a f(g(x))
2 (.) f g x = f (g x)
3
4 -- curry e uncurry (Isomorfismo A x B -> C <==> A -> (B -> C))
5 curry :: ((a, b) -> c) -> a -> b -> c
6 uncurry :: (a -> b -> c) -> (a, b) -> c
7
8 -- flip: inverte l'ordine dei parametri
9 flip :: (a -> b -> c) -> b -> a -> c
10 myFlip f a b = f b a
11
12 -- flip (:) [2, 3] 1
13 -- [1, 2, 3]
14
15 -- map e filter
16 map :: (a -> b) -> [a] -> [b] -- proprieta': map (f . g) = map f . map g
17 filter :: (a -> Bool) -> [a] -> [a]
18
19 -- zipWith: applica f. binaria agli elementi di due liste
20 zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]

```

1.5.1 Famiglia dei **fold** (schemi di ricorsione)

I **fold** astraggono il concetto di ricorsione strutturale sulle liste, sostituendo i costruttori della struttura dati (l'operatore `(:)` e la lista vuota `[]`) con funzioni e valori specifici.

Se l'operatore `#` è associativo e ha `e` come elemento neutro, allora `foldl (#) e = foldr (#) e`.

- **foldr (right fold)**: associa a destra; la riduzione avviene “al ritorno” dalle chiamate ricorsive (costruisce espressioni).

```

1 -- definizione algebrica:
2 foldr :: (a -> b -> b) -> b -> [a] -> b
3 foldr f z [] = z
4 foldr f z (x:xs) = f x (foldr f z xs)

```

- sostituisce ricorsivamente il costruttore `(:)` con la funzione `f` data e `[]` con il valore base `z`.

- espansione: `foldr f z [x1, x2, x3]` diventa `x1 'f' (x2 'f' (x3 'f' z))`
- può terminare anche su liste infinite se la funzione `f` non è stretta nel suo secondo argomento (ovvero se non ha sempre bisogno di valutare la ricorsione per restituire un risultato).

```

1  -- somma con foldr
2  foldr (+) 0 [1, 2, 3]
3  -- valutazione: 1 + (2 + (3 + 0)) = 6
4
5  -- implementazione di map usando foldr
6  myMap f xs = foldr (\x acc -> f x : acc) [] xs

```

- **foldl** (left fold): associa a sinistra; simula un ciclo iterativo passando un accumulatore “in avanti”

```

1  -- definizione algebrica:
2  foldl :: (b -> a -> b) -> b -> [a] -> b
3  foldl f z [] = z
4  foldl f z (x:xs) = foldl f (f z x) xs

```

- valuta l'espressione partendo da sinistra e procedendo verso l'interno - valuta tutto solo alla fine della lista
- espansione: `foldl f z [x1, x2, x3]` diventa `((z 'f' x1) 'f' x2) 'f' x3`
- *nota bene*: In `foldl`, la funzione prende come primo argomento l'accumulatore, e come secondo l'elemento corrente della lista.

```

1  -- sottrazione con foldl
2  foldl (-) 0 [1, 2, 3]
3  -- valutazione: ((0 - 1) - 2) - 3 = -6
4
5  -- sottrazione con foldr per confronto
6  foldr (-) 0 [1, 2, 3]
7  -- valutazione: 1 - (2 - (3 - 0)) = 2
8
9  -- reverse usando foldl
10 myReverse xs = foldl (\acc x -> x : acc) [] xs

```

- **foldr1** e **foldl1**: varianti che non richiedono un valore iniziale esplicito, poiché usano il primo o l'ultimo elemento come base; *danno errore su lista vuota*.

```

1  -- utile quando non esiste un elemento neutro sensato (es. min/max assoluto)
2  myMinimum = foldr1 min
3  myMinimum [5, 3, 8] -- risultato: 3
4  -- myMinimum []     -- genera errore a runtime

```

1.6 Alcuni esempi di stile funzionale

Lo stile funzionale evita accumulatori ed indici, preferendo la manipolazione globale delle strutture dati.

- **prodotto scalare**: può essere implementato in diversi modi (flessibilità dei funzionali):


```

1  -- accoppia gli elementi, trasforma "*" in una funzione che accetta una tupla
2  -- applica la moltiplicazione agli elementi, e somma i prodotti
3  ps1 xs ys = sum (map (uncurry (*)) (zip xs ys))
4
5  -- crea una lista di funzioni parzialmente applicate
6  -- (da [5, 4] si ottiene [(5*), (4*)])
7  -- zApp/applyL prende la lista di funzioni ^ e la applica sulla seconda lista
8  ps2 xs ys = sum (applyL (map (*) xs) ys)
9
10 -- zipWith fa zip e map in un passo solo
11 ps3 xs ys = sum (zipWith (*) xs ys)

```

- controllare se una lista è ordinata:

```

1  ordinata xs = and (zipWith (<=) xs (tail xs))

```

- **ricerca su lista** (`findVPH`): si può implementare accoppiando la lista agli indici (`zip [0..] xs`) e filtrando per il valore cercato.

Ideato da Alonzo Church nel 1931, il λ -calcolo nasce per esplicitare il *processo meccanico di calcolo* (sintassi) di una funzione, in contrasto con la visione matematica classica che vede la funzione solo come una relazione statica tra insiemi (semantica).

2.1 Sintassi e Computazione

La sintassi del λ -calcolo è minimalista ed è composta da tre regole:

- **variabile:** $x, y, z \dots$
- **astrazione:** $\lambda x.M$ (corrisponde alla definizione di una funzione con parametro x e corpo M)
- **applicazione:** (MN) (corrisponde all'applicazione della funzione M all'argomento N)

in BNF, quindi, la grammatica del lambda-calcolo è quindi:

$$T ::= V \mid \lambda V.T \mid (T T)$$

Regole di associazione

- **l'applicazione associa a sinistra:** $FN_1N_2 \dots N_n$ equivale a $(\dots (FN_1) \dots N_n)$
- **l'astrazione associa a destra:** $\lambda x_1 \dots x_n.M$ equivale a $\lambda x_1.(\lambda x_2.(\dots (\lambda x_n.M) \dots))$

La computazione avviene tramite la β -**riduzione** (beta-regola): un termine della forma $(\lambda x.M)N$ si dice *redex* e si riduce sostituendo tutte le occorrenze di x in M con il termine N . Formalmente:

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x]$$

Un termine che non contiene più β -redex è detto in **forma normale** e rappresenta un valore finale (il risultato della computazione).

2.2 Variabili e Combinatori

- **Variabili libere (FV) e legate:** in $\lambda x.M$, la variabile x è *legata* (bound) all'astrazione. Qualsiasi variabile non legata è *libera* (free).
- **α -conversione (alfa-regola):** per evitare la cattura accidentale di variabili libere durante la sostituzione, si possono rinominare le variabili legate (es. $\lambda x.x \equiv \lambda y.y$).

- è buona pratica seguire la “convenzione di Barendregt”, che stabilisce che, all’interno di un termine lambda o di un insieme di termini, tutte le variabili legate devono avere nomi distinti tra loro e diversi da tutte le variabili libere presenti nel termine stesso
- I termini tali che $(FV(M) = \emptyset)$ (λ -termini chiusi), ovvero senza variabili libere, sono chiamati **combinatori**

2.2.1 Combinatori noti e strutture di controllo

Il λ -calcolo puro non ha tipi di dato predefiniti: tutto (booleani, numeri, if-then-else) deve essere codificato tramite funzioni.

- **Identità (I):** $I \equiv \lambda x.x$
- **Cancellatori (booleani):**
 - True (K):** $K \equiv \lambda xy.x$ (prende due argomenti e restituisce il primo)
 - False (O):** $O \equiv \lambda xy.y$ (prende due argomenti e restituisce il secondo)
- **If-Then-Else:** grazie alla definizione dei booleani, la struttura condizionale è codificata come un’applicazione: $\text{if } x \text{ then } M \text{ else } N \equiv xMN$. Se x è True (K) selezionerà M , se è False (O) selezionerà N .
- **Compositore (S):** $S \equiv \lambda xyz.xz(yz)$ (prende due funzioni, x e y , e applica y a z e x al risultato dell’applicazione, ottenendo $x.y$)
(si può dimostrare che $SKK \approx I$).
- **Duplicatori e divergenza:**
 - $\omega \equiv \lambda x.xx$ (auto-applicazione)
 - $\Omega \equiv (\lambda x.xx)(\lambda x.xx)$, ovvero $\omega\omega \rightarrow \omega\omega \rightarrow \dots$ rappresenta la **funzione indefinita**, il loop infinito (non tipabile)

2.2.1.1 Numeri di Church e iteratori

I **Numerali di Church** rappresentano i numeri naturali come funzioni di ordine superiore che *iterano* un’operazione. Il numero n è una funzione che prende una funzione (es. successore) s e un valore di partenza (es. zero) z , e applica s ad z esattamente n volte

$$\underline{n} \equiv \lambda sz.s(s(\dots(sz)\dots)) \quad [n \text{ applicazioni}]$$

- $\underline{0} \equiv \lambda sz.z$ (equivale a False (K))
- $\underline{1} \equiv \lambda sz.sz$
- $\underline{2} \equiv \lambda sz.s(s(z))$
- $\underline{3} \equiv \lambda sz.s(s(s(z)))$

Per calcolare il **successore** (+1) di un numero di Church, basta creare una funzione che applica s una volta in più: $\text{succ} \equiv \lambda xsz.s(xsz)$ (o anche $\lambda xsz.xs(s(z))$)

I numerali di Church non sono quindi solo dati, ma **iteratori intrinseci**.

2.3 Ricorsione e Combinatori di Punto Fisso

Per definire funzioni ricorsive complesse il λ -calcolo necessita di un principio di ricorsione generale.

Thm. 1: Combinatore di Punto Fisso

Nel λ -calcolo esiste un termine Y (detto **Combinatore di Punto Fisso**) tale che, per ogni termine M , si ha $YM \rightarrow M(YM)$.

Il combinatore permette di trovare quindi il punto fisso di una funzione (quel valore x tale che $f(x) = x$)

Due combinatori di punto fisso sono:

- **Combinatore di Turing (Y_T):** definito come $Y_T = \theta\theta$ dove $\theta \equiv \lambda xy.y(xxy)$.

$$\begin{aligned} Y_T M &\equiv (\theta\theta)M \\ &\equiv ((\lambda xy.y(xxy))\theta)M \\ &\rightarrow_\beta (\lambda y.y(\theta\theta y))M \\ &\rightarrow_\beta M(\theta\theta M) \\ &\equiv M(Y_T M) \end{aligned}$$

- **Combinatore di Curry (Y_C):** definito come $Y_C \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$. Se applicato all'identità I , questo combinatore riduce a Ω (divergenza).

$$\begin{aligned} Y_C M &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))M \\ &\rightarrow_\beta (\lambda x.M(xx))(\lambda x.M(xx)) \\ &\rightarrow_\beta M((\lambda x.M(xx))(\lambda x.M(xx))) \\ &\equiv M(Y_C M) \end{aligned}$$

(l'ultima equivalenza $\equiv$ vale perché $(\lambda x.M(xx))(\lambda x.M(xx))$ è la forma ridotta di $Y_C M$)

Thm. 2: Tesi di Church-Turing

Ogni funzione calcolabile è λ -definibile.

Temi avanzati di programmazione funzionale

3.1 Fusion law per foldr

La **Fusion Law** per `foldr` è un principio di induzione generalizzato che permette di trasformare la composizione di una funzione con una `foldr` in un'unica `fold`. Essa stabilisce le condizioni per cui vale l'uguaglianza:

$$f \cdot \text{foldr } g \ a = \text{foldr } h \ b$$

L'obiettivo è quindi trovare funzioni o valori `h` e `b` tali che l'applicazione di `f` al risultato di una `foldr` sia equivalente ad una nuova `foldr` che lavora direttamente su di essi - l'utilità computazionale risiede nella capacità di "fondere" due passaggi (una trasformazione `f` applicata dopo un `fold` con `g`) in un unico ciclo sulla struttura dati, evitando allocazioni intermedie.

Thm. 3: Fusion law

Siano `f`, `g`, `h` funzioni e `a`, `b` valori. L'uguaglianza $f \cdot \text{foldr } g \ a = \text{foldr } h \ b$ è garantita se sono soddisfatti i seguenti tre requisiti:

- (1) `f` è stretta ($f \text{ undefined} = \text{undefined}$) (così che valga anche nel caso di liste parziali/indefinite)
- (2) **caso base:** $f \ a = b$ - `f` applicata all'accumulatore iniziale della prima `foldr` (`a`) deve dare come risultato l'accumulatore iniziale della seconda (`b`)
- (3) **passo induttivo** (condizione di fusione): per ogni `x`, `y`, deve valere:

$$f \ (g \ x \ y) = h \ x \ (f \ y)$$
 - in questo modo, l'operazione `h` del nuovo `foldr` riproduce esattamente l'effetto di `g` seguito da `f`

parallelo con un omorfismo

La fusion law può essere interpretata come la dimostrazione che `f` agisce come un **omomorfismo** tra due strutture algebriche di calcolo:

- **preservazione dell'identità:** $f \ a = b$ mappa l'elemento neutro (accumulatore iniziale) del primo dominio in quello del secondo, analogamente a $f(1_A) = 1_B$.
- **preservazione dell'operazione:** La condizione $f(g \ x \ y) = h \ x \ (f \ y)$ ricalca la proprietà omomorfica $f(a \bullet b) = f(a) \circ f(b)$.

- *sorgente* ($g\ x\ y$): rappresenta l'operazione di combinazione di un elemento x con il risultato parziale (accumulatore) y tramite la funzione g
- *destinazione* ($h\ x\ (f\ y)$): rappresenta la nuova operazione h che combina x con il risultato già trasformato da f
- **NB:** in $h\ x\ (f\ y)$, l'elemento x non viene trasformato da f - nella struttura della lista, x è un dato "guida" di tipo A , mentre y è il risultato della ricorsione (accumulatore) di tipo B . La fusion law mira a trasformare il *risultato complessivo* del fold, non i singoli elementi contenuti nella lista.

Derivazione formale

Per dimostrare la validità dell'uguaglianza $f . foldr\ g\ a = foldr\ h\ b$, analizziamo separatamente i due lati della relazione applicando i principi di induzione sulla struttura della lista:

Ricordando la definizione di `foldr` basata sulla decomposizione della lista:

- `foldr g a [] = a`
- `foldr g a (x:xs) = g x (foldr g a xs)`

Caso [] (base):

- **sinistra:** $(f . foldr\ g\ a)\ []$
 $= f\ (foldr\ g\ a\ [])$ (*def di .*)
 $= f\ a$ (*def di foldr*)
- **destra:** `foldr h b []`
 $= b$ (*def di foldr*)
- da cui deriva la prima condizione: $f\ a = b$

Caso (x:xs) (induttivo):

- **sinistra:** $(f . foldr\ g\ a)\ (x:xs)$
 $= f\ (foldr\ g\ a\ (x:xs))$ (*def di .*)
 $= f\ (g\ x\ (foldr\ g\ a\ xs))$ (*def di foldr*)
- **destra:** `foldr h b (x:xs)`
 $= h\ x\ (foldr\ h\ b\ xs)$ (*def di foldr*)
 $= h\ x\ (f\ (foldr\ g\ a\ xs))$ (*induzione*)
- ponendo $y = foldr\ g\ a\ xs$, deriviamo la condizione di fusione:
 $f\ (g\ x\ y) = h\ x\ (f\ y)$

Per esempio, consideriamo la composizione `sum . map (*2)`.

Abbiamo che `map (*2)` si può scrivere come una `foldr` in questo modo:

`map (*2) = foldr (\x acc -> (x * 2) : acc) []`

Usiamo la fusion law per fondere `sum` e `map`:

- `sum undefined = undefined`
- `sum [] = 0` (il nostro accumulatore iniziale è 0)
- $f(g\ x\ y) = h\ x\ (f\ y)$:
 – a sinistra abbiamo $sum((x * 2) : y)$, che per definizione è $(x * 2) + sum\ y$

- a destra abbiamo $h\ x\ (sum\ y)$
- uguagliamo: $(x * 2) + sum\ y = h\ x\ (sum\ y)$
- se sostituiamo $sum\ y$ con un generico $total$, otteniamo la definizione di h : $h\ x\ total = (x * 2) + total$

Otteniamo quindi il risultato fuso:

```
1 sum . map (*2) = foldr (\x total -> (x * 2) + total) 0
```

3.2 Lazy evaluation

La valutazione lazy di Haskell è formata da tre aspetti precisi:

$$\text{WHNF} + \underbrace{\text{Call-by-name} + \text{Sharing}}_{\text{"Call-by-need"}}$$

3.2.1 Weak Head Normal Form (WHNF)

Nel λ -calcolo (e in generale), è possibile scegliere diversi insiemi come *termini* (valori che non è necessario ridurre ulteriormente).

Una scelta naturale è quella della **forme normali**, ovvero i termini che non contengono redex (e.g. $\lambda x.x$ o $\lambda xyz.xz(yz)$).

Un'altra opzione è quella delle **forme normali di testa** (*head-normal form*).

Def. 1: Head-Normal Form

Un qualsiasi termine generico nel λ -calcolo si può scrivere in questa forma strutturale:

$$\lambda x_1 \dots x_n . (M_0 M_1 \dots M_k)$$

Il termine più a sinistra nelle parentesi (M_0) si chiama *head*.

Un termine è in **head-normal form** se l'elemento in testa (M_0) è una **variabile** (libera o legata) e non è possibile fare alcuna β -riduzione in quella posizione.

Ovvero, se ha la struttura:

$$\lambda x_1 \dots x_n . (x\ M_1 \dots M_k)$$

(dove x è una variabile, e gli M_i non sono necessariamente in forma normale) (non è necessario che ci sia una lambda astrazione iniziale, anche $x\ M_1 \dots M_k$ è in HNF).

(se il termine è un termine chiuso - i.e. senza variabili libere - x deve essere una delle variabili legate dal λ)

Ovvero, un termine è in hnf se la "testa" della computazione è stabile.

Un'altra alternativa è data dalla whnf, versione più lazy:

Def. 2: Weak-Head-Normal Form

Un termine è in **weak-head-normal form** se si trova in uno di questi due stati:

- è in HNF (quindi ha una variabile in testa), oppure
- inizia con una λ -astrazione esterna, indipendentemente da cosa c'è dentro

Ovvero, se ha una di queste due strutture:

- $x M_1 \dots M_k$
- $\lambda x.M$

Alcuni esempi

Dato $\Omega = (\lambda x.xx)(\lambda x.xx)$:

- $\lambda e.\Omega$ è in WHNF (inizia con λe) ma non in HNF (dentro il lambda la head è Ω , che non è una variabile)
- $x \Omega$ è in WHNF e anche in HNF (la testa è una variabile)
- Ω non è in WHNF e neanche in HNF (non è un λ e non inizia con una variabile)

Quindi, per Haskell, un'espressione viene considerata un “valore” quando raggiunge la whnf (ovvero quando il guscio più esterno è un blocco invalutabile).

Perciò:

- una funzione pura è un valore (è un λ)
- se l'espressione è un'applicazione di funzione, Haskell sostituisce il nome della funzione con il suo corpo interno (la sua “definizione”) e continua a ridurre l'espressione fino a quando non riesce ad “estrarre” un λ all'esterno
 - per esempio, usiamo la funzione `id` per definire una nuova funzione:

```
1 creaAdd :: Int -> (Int -> Int)
2 creaAdd x = id (\y -> x + y)
```

- e proviamo a valutare l'espressione `creaAdd 5`
- Haskell vede `creaAdd 5`; per prima cosa sostituisce il nome della funzione con il suo corpo applicando l'argomento: `id (\y -> 5 + y)`
- questa espressione non è ancora in whnf (perché è un'applicazione)
- il sistema valuta quindi il redex più esterno (l'applicazione di `id`); riduce quindi a `\y -> 5 + y`
- questa espressione è in whnf, quindi Haskell smette di valutarla

Per quanto riguarda i **tipi di dato**, invece, Haskell li definisce come in whnf se il costruttore di dati più esterno è visibile e non può essere ulteriormente ridotto.

Per esempio, nel caso delle liste (definite tramite i costruttori `:` e `[]`): una lista è in whnf se è nella forma `x:xs`, indipendentemente dal fatto che `x` e `xs` siano valutate.

Per esempio, sono in whnf sia `(3+2) : merge [1,2,3] [4, 5,6]` che `undefined : undefined`.

3.2.2 Call-by-name (CBN)

I linguaggi imperativi usano spesso la *call-by-value*. In CBV (eager), gli argomenti di una funzione vengono valutati prima che l'esecuzione entri nella funzione stessa (per più informazioni, vedi anche appunti del corso “Linguaggi di programmazione”).

La **Call-By-Name** ribalta questo funzionamento:

- gli argomenti vengono passati alla funzione come espressioni non ancora valutate
- le espressioni vengono valutate solo quando (e se !!) i loro valori sono necessari
- la selezione dell'ordine di calcolo segue la strategia *leftmost-outermost*: il motore di runtime valuta sempre il *redex* situato più all'esterno e più a sinistra

Per esempio, se consideriamo questo caso:

```
1 constFun x = 3
2
3 loop x = loop x -- computazione infinita
4
5 -- vogliamo valutare:
6 > constFun (loop 1)
```

- se si valuta `constFun (loop 1)` in CBV, il sistema cercherà di valutare `loop 1`, causando un ciclo infinito
- se si valuta in CBN, invece, l'espressione rimane in-valutata; poiché il corpo non la usa, la funzione restituirà semplicemente `3`

La Call-By-Name nativa ha però un difetto: **non ha memoria**.

Se definiamo qualcosa come `sqr x = x * x` e proviamo a valutare `sqr (3+4)`, otteniamo `sqr (3+4) * (3+4)`: nonostante si tratti dello stesso `x`, l'argomento viene duplicato. Per ottenere il risultato dobbiamo quindi calcolare `3 + 4` due volte.

Haskell risolve questo problema introducendo lo **sharing**.

Def. 3: Call-by-need

La combinazione di Call-by-name e sharing viene chiamata **Call-by-need**.

3.2.3 Sharing

Per evitare il difetto di duplicazione della CBN, Haskell rappresenta i **thunk** (sospensioni di calcolo) in memoria non come alberi, ma come **DAG** (grafi diretti aciclici).

Quando un argomento viene passato a una funzione, Haskell non copia l'espressione in due luoghi distinti in memoria, ma crea due **puntatori che puntano allo stesso thunk** (gli argomenti verranno quindi valutati una volta sola).

In questo modo, sappiamo con certezza che una valutazione lazy **non fa mai più riduzioni di una stessa valutazione eager**. Può invece farne anche molte di meno!

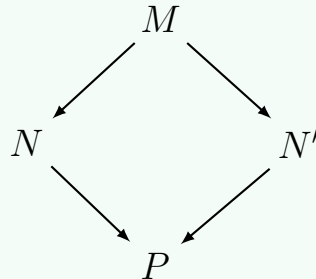
Però, può lasciare molte sottoespressioni non valutate, occupando a volte molto spazio in memoria.

Il teorema che ci assicura che la valutazione lazy sia corretta è il seguente:

Thm. 4: Teorema di confluenza [Church-Rosser]

Se un'espressione può essere ridotta in due modi diversi arrivando a risultati diversi, esiste sempre una "strada comune" per farli ricongiungere.

Formalmente: se $M \rightarrow N$ e $M \rightarrow N'$, allora esiste un termine P tale che $N \rightarrow P$ e $N' \rightarrow P$.



Il fatto che P esista non vuol dire che sia facile trovarlo. Tuttavia, se tutte le riduzioni di M terminano, allora necessariamente P è un valore e viene raggiunto da tutte le riduzioni.

Corollary for Thm. 4: Unicità delle forme normali

Se N e N' sono forme normali e $M \rightarrow N$ e $M \rightarrow N'$, allora $N = N'$

Non è detto che le forme normali vengano trovate, perché ci potrebbero essere computazioni non terminanti.

Un esempio canonico è il termine $K\ I\ \Omega$.

Con la CBV, si proverebbe a ridurre Ω , e si andrebbe avanti all'infinito. Con la CBN, invece, si applica prima la funzione a sinistra (leftmost-outermost): poiché K restituisce solo il primo argomento, ignora completamente la valutazione di Ω . Con un solo passo si ottiene il risultato finale: I .

Questo comportamento è formalizzato dal seguente teorema:

Thm. 5: Teorema di standardizzazione

Se per un termine M esiste una riduzione $M \rightarrow P$ e P è in forma normale, allora la computazione call-by-name (leftmost-outermost) è tale per cui $M \rightarrow_{cbn} P$ in un numero finito di passi.

Ovvero, se esiste una riduzione terminante, tra quelle che terminano c'è sicuramente la CBN.

3.3 Funzioni non strette

Nei linguaggi imperativi, tutte le funzioni di base sono strette (**strict**): se viene passato loro un valore che va in loop infinito (o crasha), l'intero programma fallisce.

In Haskell, grazie alla valutazione lazy, si possono definire funzioni non strette.

Def. 4: Funzione non stretta

Una funzione si dice non stretta se è in grado di restituire un valore anche quando la valutazione di alcuni dei suoi parametri potrebbe non terminare o generare un errore.

(Una funzione è stretta se `f(undefined) = undefined`.)

Un esempio molto semplice è il “cancellatore” `K`:

```
1 K = \x y -> x
2
3 omega x = omega x -- loop infinito
4
5 > K id (omega 4) 3
6 > 3 -- valuta a 3 perché K non dipende dal suo secondo argomento
```

Esempio: `null`

Consideriamo la funzione predefinita `null`, che verifica se una lista sia vuota.

Per sapere se una lista è vuota, ad Haskell non serve calcolare l'intera lista o i suoi elementi: basta fare pattern-matching per vedere se la lista è in WHNF, ovvero se mostra il costruttore `:` o `[]`.

```
1 myNull [] = True
2 myNull _ = False
3
4 > myNull [undefined]
5 > False
6
7 > myNull (undefined:undefined)
8 > False
9
10 > myNull undefined
11 > *** Exception: Prelude.undefined
```

Esempio: varianti di OR

I linguaggi imperativi implementano nativamente la “valutazione a corto circuito” per gli operatori logici; ad esempio,

```
if (x == 0 || mod(y, x) == 2)
```

non genererà mai un errore di divisione per zero quando `x == 0`, perché la prima condizione valuta a `True` e il sistema termina saltando la seconda espressione.

Tuttavia, a causa della politica call-by-value di C, è impossibile definire una funzione personalizzata:

```
int myor(int x, int y)
```

che si comporti esattamente come l'operatore `||`. La call-by-value, infatti, forzerà sempre e comunque la valutazione preliminare di entrambi i parametri prima di effettuare la chiamata. In Haskell, al contrario, la natura lazy del linguaggio ci permette di farlo senza problemi.

Possiamo scrivere diverse definizioni della funzione OR in Haskell per analizzarne la tolleranza verso espressioni non valutabili (`undefined`):

```

1  -- 1. left OR (valuta prima il primo parametro)
2  leftor :: Bool -> Bool -> Bool
3  leftor True _ = True
4  leftor False x = x
5
6  -- 2. eager OR (forza la valutazione di entrambi)
7  eagerOr :: Bool -> Bool -> Bool
8  eagerOr True True = True
9  eagerOr False True = True
10 eagerOr True False = True
11 eagerOr False False = False
12
13 -- 3. which OR (basato sull'ordine dei pattern)
14 whichor :: Bool -> Bool -> Bool
15 whichor False False = False
16 whichor _ _ = True
17
18 -- 4. right OR (definito tramite inversione dei parametri)
19 rightor :: Bool -> Bool -> Bool
20 rightor = flip leftor

```

Interrogando l'interprete con il valore `undefined`, scopriamo che queste definizioni apparentemente equivalenti mostrano comportamenti semantici diversi:

- `leftor True undefined → True`: il primo argomento soddisfa la clausola `leftor True _`, quindi il secondo argomento viene ignorato e non causa l'eccezione
- `eagerOr True undefined → *** Exception: Prelude.undefined`: la definizione esplicita costringe il runtime a risolvere il valore di entrambi i parametri per decidere quale riga applicare
- `whichor True undefined → True`: questo caso dipende dall'ordine di valutazione dei pattern: l'interprete analizza la prima riga (`False False`), non trova un match e scende immediatamente alla seconda riga, che cattura l'espressione restituendo `True` senza toccare il secondo argomento
- `rightor undefined True → True`: come il simmetrico di `leftor`, questa variante riesce a cortocircuitare a `True` se l'argomento valido si trova a destra

Sarebbe ideale poter disporre di un Parallel OR (`por`), una funzione definita in modo tale da soddisfare contemporaneamente le seguenti equazioni:

```

1  por undefined True == True
2  por True undefined == True

```

In Haskell standard, purtroppo, questo non è realizzabile. L'interprete fissa un ordine di valutazione sequenziale rigoroso: esplora i pattern dall'alto verso il basso e i parametri da sinistra a destra (a meno di pattern non stringenti come le variabili libere). Per ottenere il comportamento del `por` (noto teoricamente come l'operatore *por* del sistema PCF), il runtime dovrebbe eseguire le due espressioni concorrentemente in parallelo, fermandosi non appena una delle due termina con successo e congelando o bloccando l'eccezione dell'altra.

3.4 Tecniche di ottimizzazione

3.4.1 Impatto della Laziness sulla Complessità Asintotica

Nei linguaggi a valutazione eager, la complessità asintotica della composizione di due funzioni $f \cdot g$ applicate a un input x è data dalla somma del costo necessario a valutare l'argomento sommato al costo della funzione esterna: $T(f \cdot g)|x| = T(f)(|g(x)|) + T(g)|x|$.

Nei linguaggi lazy questo principio cessa di essere valido: la funzione esterna f potrebbe richiedere solo una valutazione parziale del risultato di g , o dipendere esclusivamente da una frazione microscopica dei dati generati da $g(x)$. Poiché non esiste ancora una teoria formale onnicomprensiva in grado di modellare analiticamente la complessità lazy, si adotta convenzionalmente il modello eager come *upper bound* per stimare il comportamento dei programmi.

Esempio: `head . insSrt`

Un esempio di come la valutazione lazy alteri la complessità di una computazione consiste nel determinare il valore minimo di una lista effettuando una chiamata alla funzione `head` su una lista preventivamente ordinata tramite l'algoritmo Insertion Sort (`insSrt`):

```

1 ins :: Ord a => a -> [a] -> [a]
2 ins y [] = [y]
3 ins y (x:xs)
4   | y <= x    = y : x : xs
5   | otherwise = x : ins y xs
6
7 insSrt :: Ord a => [a] -> [a]
8 insSrt [] = []
9 insSrt (x:xs) = ins x (insSrt xs)
10
11 minimo :: Ord a => [a] -> a
12 minimo = head . insSrt

```

In un paradigma d'esecuzione di tipo eager, l'ordinamento completo della lista verrebbe eseguito in ogni caso, imponendo all'intera computazione un costo quadratico pari a $\Theta(n^2)$.

In Haskell, la funzione `head` esige la valutazione del suo argomento esclusivamente fino al raggiungimento della Weak Head Normal Form, la quale coincide con la determinazione del primo costruttore di cella `:` e del relativo elemento di testa. Durante l'elaborazione, la funzione di inserimento ordinato `ins` viene srotolata unicamente per la frazione di passi strettamente necessaria a identificare e spingere l'elemento minimo assoluto in cima alla lista. La pigrizia del runtime interrompe il calcolo del resto dell'ordinamento non appena la testa è visibile. Di conseguenza, la valutazione lazy converte implicitamente la semantica di un *Insertion Sort* in quella di un *Selection Sort*, facendo crollare la complessità asintotica dell'operazione da quadratica a lineare $\Theta(n)$.

3.4.1.1 Compromesso Spazio-Tempo: powerset

Al fine di preservare la trasparenza referenziale e l'assenza di effetti collaterali, i dati in Haskell sono strutturalmente immutabili. L'impossibilità di modificare le strutture *in-place* comporta la necessità di effettuare copie e allocazioni continue nell'heap, costringendo il programmatore funzionale a valutare attentamente il bilanciamento tra tempo di computazione e occupazione di memoria, come dimostrato dal calcolo dell'insieme delle parti (`powerset`).

Una prima implementazione ottimizza il tempo di calcolo memorizzando il risultato ricorsivo all'interno di una clausola locale `where` :

```
1 powerset :: [a] -> [[a]]
2 powerset []      = [[]]
3 powerset (x:xs) = ps ++ map (x:) ps
4   where ps = powerset xs
```

Questa formulazione previene il ricalcolo di espressioni identiche ed esegue un numero di riduzioni temporalmente ottimo pari a $\Theta(2^n)$. Tuttavia, dal punto di vista dello spazio, la lista `ps` deve essere integralmente preservata in memoria lungo l'intera discesa ricorsiva per consentire la successiva valutazione della parte destra dell'espressione, generando un carico di ritenzione elevato nella heap.

Al contrario, è possibile implementare l'algoritmo effettuando due chiamate ricorsive esplicite e separate:

```
1 powersetLento :: [a] -> [[a]]
2 powersetLento []      = [[]]
3 powersetLento (x:xs) = powersetLento xs ++ map (x:) (powersetLento xs)
```

In questa seconda versione, l'occupazione di spazio si riduce perché le sotto-liste possono essere deallocate dal garbage collector non appena consumate. Il prezzo da pagare si traduce in un incremento del tempo di calcolo, che sale a una complessità asintotica di $\Theta(n2^n)$ a causa della totale duplicazione dell'albero delle chiamate ricorsive.

3.4.1.2 Memoization e Scope delle variabili

Per massimizzare il riutilizzo di strutture dati complesse o infinite, Haskell adotta una strategia di **memoization** (memorizzazione dei thunk già risolti). La persistenza di questa memoria non è tuttavia illimitata, ma è rigidamente governata dal livello di dichiarazione (*scope*) delle espressioni:

- **variabili top-level**: se una struttura dati viene definita al livello più esterno del modulo, una sua porzione valutata durante una computazione viene conservata stabilmente in memoria per l'intera durata del programma. Qualsiasi invocazione successiva accederà ai dati già calcolati a costo zero, migliorando i tempi a scapito di un consumo fisso di spazio.
- **definizioni locali**: se la stessa struttura viene dichiarata internamente a una funzione mediante un blocco `where` o `let`, il runtime provvederà a scaricarla e deallocarla completamente non appena l'esecuzione di quel blocco termina. Questa pulizia previene la saturazione della memoria, ma costringe il sistema a ricalcolare interamente l'espressione da zero a ogni successiva chiamata della funzione.

3.4.2 Parametri Accumulatori

Nella programmazione funzionale, l'allocazione e la deallocazione della memoria avvengono in modo implicito e, per preservare la trasparenza referenziale, i dati sono immutabili. Di conseguenza, la scrittura naïve di alcuni algoritmi ricorsivi può comportare continue copie di strutture dati o un numero ridondante di computazioni, inficiando l'efficienza in tempo e spazio.

Una delle opzioni per ottimizzare le funzioni ricorsive consiste nell'uso dei **parametri accumulatori**: le informazioni possono essere memorizzato all'interno di parametri ausiliari condivisi tra le varie chiamate ricorsive, emulando la presenza di un piccolo "stato" locale.

Esempio: Fibonacci

Il calcolo della successione di Fibonacci basato direttamente sulla definizione matematica è un classico caso di funzione inefficiente:

```
1 fib :: Int -> Int
2 fib 0 = 1
3 fib 1 = 1
4 fib n = fib (n-1) + fib (n-2)
```

A causa delle doppie chiamate ricorsive ridondanti, questo algoritmo richiede un numero di somme esponenziale, con una complessità di $\Theta(\phi^n)$ (dove ϕ è la sezione aurea).

Possiamo linearizzare il calcolo introducendo una funzione di supporto `fibAux` che sfrutta due parametri accumulatori per memorizzare, a ogni passo, gli ultimi due numeri di Fibonacci calcolati:

```
1 fibE :: Int -> Int
2 fibE n = fibAux n 2 0 1
3   where
4     fibAux n i f fp
5       | i == n    = f
6       | otherwise = fibAux n (i+1) (f+fp) f
```

In questo modo, lo stato del programma iterativo viene portato in avanti durante la ricorsione, riducendo la complessità del calcolo a un tempo lineare $\Theta(n)$.

3.4.3 Tupling

Il **Tupling** è una tecnica di ottimizzazione che consiste nel far restituire a una funzione una tupla (tipicamente una coppia) contenente più informazioni correlate, calcolate simultaneamente durante un'unica scansione di una struttura dati. Questo approccio consente di evitare passate multiple sulla stessa risorsa, riducendo sia i tempi di calcolo sia il numero di allocazioni in memoria.

Esempio: Fibonacci 2

Oltre all'uso dei parametri accumulatori che spingono lo stato in avanti, il calcolo efficiente dei numeri di Fibonacci può essere implementato facendo calcolare alla funzione ricorsiva gli ultimi due valori della successione, restituendoli all'indietro sotto forma di coppia:

```
1 fib :: Int -> Int
2 fib = fst . fibAux
3   where
4     fibAux 0 = (0, 1)
5     fibAux n = (f1 + f2, f1)
6       where
7         (f1, f2) = fibAux (n - 1)
```

3.4.3.1 Tupling con `foldr`

Dal punto di vista formale, quando dobbiamo determinare due proprietà distinte sulla stessa lista mediante due funzioni `foldr` indipendenti, è possibile fondere le due scansioni in un unico passaggio combinato grazie al tupling:

$$(\text{foldr } f \ a \ xs, \text{foldr } g \ b \ xs) = \text{foldr } h \ (a, b) \ xs$$

dove la funzione di accumulo combinata `h` è:

$$h \ x \ (y, z) = (f \ x \ y, g \ x \ z)$$

Questa equivalenza matematica non è valida per liste parziali o indefinite. In Haskell, infatti, vale che:

$$\text{undefined} \neq (\text{undefined}, \text{undefined})$$

Mentre l'espressione `undefined` pura rappresenta un fallimento totale del calcolo (non è in WHNF), la coppia `(undefined, undefined)` è a tutti gli effetti un valore valido in WHNF, poiché il costruttore di coppia `(,)` al livello più esterno è visibile ed è già stato estratto. Di conseguenza, le funzioni che effettuano pattern matching sulla struttura esterna avranno comportamenti differenti nei due casi.

3.4.4 Strictness e gestione dello spazio

La valutazione lazy, sebbene prevenga computazioni superflue, introduce il rischio di accumulare sospensioni di calcolo (*thunk*) non valutate nella heap. Questo fenomeno è particolarmente evidente nell'uso di determinati funzionali ricorsivi e prende il nome di *space leak*.

3.4.4.1 `foldl` e `seq`

Il funzionale `foldl` è associativo a sinistra. Per sua natura, esso non estrae alcuna informazione utile durante la scansione della lista, limitandosi ad accumulare un'espressione nidificata di applicazioni funzionali che viene risolta solo dopo aver interamente esaurito la struttura:

$$\begin{aligned} \text{foldl } (+) \ 0 \ [1..4] &= \text{foldl } (+) \ (0+1) \ [2..4] \\ &= \text{foldl } (+) \ ((0+1)+2) \ [3..4] \\ &= \text{foldl } (+) \ (((0+1)+2)+3) \ [4] \\ &= \text{foldl } (+) \ (((((0+1)+2)+3)+4) \ [] \\ &= (((((0+1)+2)+3)+4) \\ &= 10 \end{aligned}$$

Per una lista di lunghezza n , questo comportamento determina un consumo di spazio lineare $\Theta(n)$ nell'heap per memorizzare l'albero delle espressioni prima di avviare l'effettiva riduzione numerica.

Per ovviare a questo spreco di memoria, Haskell introduce una primitiva speciale non definibile all'interno del linguaggio stesso:

```
1 seq :: a -> b -> b
```

L'espressione `seq x y` forza la valutazione dell'argomento `x` fino alla WHNF prima di procedere con la valutazione e la restituzione del secondo argomento `y`.

Sfruttando questa primitiva, la libreria `Data.List` definisce il funzionale strict `foldl'`, il quale forza la riduzione dell'accumulatore parziale a ogni singolo passo della ricorsione:


```

1 foldl' :: (b -> a -> b) -> b -> [a] -> b
2 foldl' f v [] = v
3 foldl' f v (x:xs) = y 'seq' foldl' f y xs
4   where y = f v x

```

Grazie a `seq`, l'accumulatore viene costantemente ridotto, mantenendo il consumo di spazio rigidamente costante $\Theta(1)$ per tutta la durata del ciclo ricorsivo.

Esempio: `foldl' + seq`

Non sempre però il solo utilizzo di `foldl'` basta ad evitare space leaks.

Consideriamo il problema di calcolare la media degli elementi di una lista. Per effettuare il calcolo in un'unica passata ed evitare di scorrere la lista due volte (con `sum` e `length`), uniamo le due operazioni in una coppia applicando il funzionale `foldl'`:

```

1 meanIngenuo :: [Float] -> Float
2 meanIngenuo xs = s / fromIntegral l
3   where
4     (s, l) = foldl' (\(accS, accL) x -> (accS + x, accL + 1)) (0, 0) xs

```

Nonostante l'utilizzo di `foldl'` — che teoricamente è progettato per evitare la pigrizia forzando l'accumulatore ad ogni passo — questo codice continua ad accumulare espressioni non valutate (*thunk*), fallendo l'obiettivo di operare in spazio costante $\Theta(1)$.

Il motivo risiede nel fatto che `foldl'` riduce l'accumulatore solo fino alla WHNF. Per una tupla nella forma $(f\ x, g\ x)$, trovarsi in WHNF significa semplicemente che il runtime verifica la presenza del costruttore di coppia esterno $(,)$ e arresta immediatamente la riduzione, senza scendere a valutare le espressioni numeriche racchiuse all'interno.

A ogni iterazione sulla lista, la struttura in memoria si evolve quindi in questo modo:

- passo 1: $(0 + x_1, 0 + 1)$
- passo 2: $((0 + x_1) + x_2, (0 + 1) + 1)$
- passo 3: $((((0 + x_1) + x_2) + x_3, ((0 + 1) + 1) + 1)$

Invece di calcolare i valori correnti, Haskell accumula una catena di somme sospese dentro la tupla che cresce linearmente con la lunghezza della lista, causando potenzialmente uno stack overflow.

Per risolvere il problema e garantire un consumo di spazio costante $\Theta(1)$, bisogna spingere manualmente la valutazione fino alla profondità necessaria. Usando l'operatore `seq`, possiamo forzare la riduzione dei singoli elementi numerici della coppia prima di procedere con l'iterazione successiva:

```

1 meanEfficiente :: [Float] -> Float
2 meanEfficiente xs = s / fromIntegral l
3   where
4     (s, l) = foldl' f (0, 0) xs
5     f (accS, accL) x = accS 'seq' accL 'seq' (accS + x, accL + 1)

```

3.4.4.2 Divergenza tra `foldl` e `foldl'`

Sebbene su funzioni strettamente numeriche `foldl` e `foldl'` producano lo stesso identico risultato, essi possono divergere semanticamente nel caso in cui la funzione foldata **non sia stretta** sul suo primo argomento.

Consideriamo la seguente funzione non stretta:

```
1 f :: Int -> Int -> Int
2 f acc x = if x == 0 then undefined else 0
```

Valutando l'espressione `foldl f 0 [0, 2]` e la sua controparte strict `foldl' f 0 [0, 2]`, si ottengono due esiti differenti:

- `foldl f 0 [0, 2] → 0`: la riduzione procede in modo lazy ed espande l'espressione in `f (f 0 0) 2`. Poiché il secondo argomento della chiamata esterna è `2` ($\neq 0$), la condizione dell'`if` fallisce e la funzione esterna restituisce immediatamente `0`, ignorando completamente l'argomento interno `(f 0 0)` grazie alla valutazione non stretta.
- `foldl' f 0 [0, 2] → *** Exception: Prelude.undefined`: il meccanismo di `seq` forza la valutazione dell'accumulatore intermedio alla prima iterazione, calcolando preventivamente `f 0 0`. Poiché in questo passo il secondo argomento è `0`, l'espressione collassa immediatamente in `undefined`, propagando l'eccezione e interrompendo l'intera esecuzione.

3.4.5 Zucchero Sintattico: `$` e `$!`

Haskell fornisce due operatori infissi per effettuare l'applicazione di funzioni riducendo l'uso delle parentesi esterne:

- `$` (applicazione lazy): definito come `f $ x = f x`, mantiene l'argomento non valutato.
- `$!` (applicazione eager): definito come `f $! x = x 'seq' f x`, forza la valutazione dell'argomento a WHNF prima di applicarvi la funzione `f`.

3.4.6 `scanl` e `scanr`

`scanl` e `scanr` sono due funzionali che permettono di processare una lista restituendo non solo il risultato finale (come fa un `fold`), ma una lista contenente tutti i **risultati intermedi** della computazione.

Come `foldr` e `foldl`, i due funzionali si distinguono per la direzione in cui accumulano i risultati:

- `scanl` calcola i risultati “in avanti”, applicando la funzione a tutti i **prefissi** della lista:

```
1 scanl (#) v [x, y, z] = [v, v#x, (v#x)#y, ((v#x)#y)#z]
2
3 scanl (+) 0 [1, 2, 3] = [0, 1 (ovvero 0+1), 3 (0+1+2), 6 (0+1+2+3)]
```

- `scanr` calcola i risultati “all'indietro”, applicando la funzione a tutti i **suffissi** della lista

```
1 scanr (#) v [x, y, z] = [x#(y#(x#e)), y#(z#e), (z#v), v]
2
3 scanr (+) 0 [1, 2, 3] = [6 (3+2+1), 5 (3+2), 3, 0]
```

Derivazione del codice di `scanl`

La specifica formale di `scanl` definisce il funzionale come l'applicazione di un `foldl` a tutti i segmenti iniziali (`inits`) di una lista:

```
1 scanl f e = map (foldl f e) . inits
```

Tuttavia, questa definizione non è efficiente (porta ad un comportamento quadratico). Tramite la program calculation, è possibile derivare una definizione ricorsiva diretta, di complessità lineare ($\Theta(n)$).

Le equazioni che definiscono la versione efficiente (lineare) di `scanl` sono le seguenti:

```
1 scanl f e [] = [e]
2 scanl f e (x:xs) = e : scanl f (f e x) xs
```

In questa forma, il risultato dell'applicazione della funzione (`f e x`) viene passato come nuovo valore iniziale per la chiamata ricorsiva sulla coda della lista, evitando di ricalcolare i prefissi da zero.

3.5 Strutture dati infinite

La valutazione lazy permette di maneggiare in modo astratto ed elegante **strutture dati infinite**.

Gli abitanti di un tipo ricorsivo in Haskell sono la **massima algebra chiusa** rispetto all'applicazione dei costruttori.

Def. 5: Massima vs minima algebra chiusa

- *Minima algebra chiusa* (induzione): insieme di tutti i dati finiti che si possono costruire partendo dai casi base (esempio: numeri naturali di Peano)
- *Massima algebra chiusa* (co-induzione): include anche gli oggetti costruiti applicando i costruttori infinite volte - i tipi di Haskell ammettono questa natura infinita
per esempio, la co-algebra dei naturali includerebbe anche il 'naturale infinito'

$$\omega = S(S(\dots)) = S^\infty(\dots)$$

Nel caso delle liste, le liste *finite* sono la minima algebra chiusa rispetto a `:` e `[]`, mentre le liste infinite (o **stream**) sono la relativa massima algebra.

Eliminando i costruttori costanti (casi base) dalle algebre induttive, si possono "isolare" gli elementi infiniti:

- eliminando `zero` dai naturali, si ottiene l'algebra che contiene solo il naturale infinito
- eliminando `[]` dalle liste, si ottiene l'algebra che contiene solo stream

3.5.1 Definizioni naive e circolari

Il modo che si sceglie per generare stream ne cambia drasticamente l'efficienza.

Possiamo definire le implementazioni di stream in due categorie: naif (basate sul ricalcolo), e circolari (basate sulla condivisione)

3.5.1.1 Definizioni naif

Una definizione naif richiama esplicitamente se stessa passando nuovamente gli argomenti

esempio: potenze e iterate

```
1 powers n = 1 : map (n*) (powers n)
2
3 myIterate f x = x : map f (myIterate f x)
```

Queste implementazioni hanno complessità quadratica ($O(n^2)$), in quanto, per calcolare l' n -esimo elemento, il programma deve ripercorrere tutti i passaggi precedenti.

La complessità è causata da un'assenza di memoizzazione del lavoro già fatto: ogni volta che si chiede un nuovo elemento viene generata una nuova chiamata alla funzione, creando una catena infinita di funzioni annidate (`map (*2) (map (*2) (map (*2) ...))`).

3.5.1.2 Definizioni circolari e lineari

Una definizione si dice circolare quando la struttura dati viene definita in termini di se stessa, creando un puntatore in memoria (Graph Reduction) invece di generare nuove chiamate a funzione. Questo garantisce una complessità lineare ($O(n)$) perché ogni elemento viene calcolato una volta sola.

Esistono due modi principali per implementarle:

- **livello globale:** senza parametri, il nome della lista è sufficiente per creare il riferimento circolare

```
1 ones = 1 : ones
2 fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

- **all'interno di funzioni:** si usano `where` o `let` per “fissare” i parametri e definire la lista ricorsivamente rispetto alla variabile locale

```
1 powers' n = ps where ps = 1 : map (*n) ps
2 myIterate' f x = xs where xs = x : map f xs
```

È utile saper distinguere tra velocità (linearità) e struttura di memoria (circolarità). Non tutte le ricorsioni dirette sono inefficienti.

```
1 iterate1 f x = x : iterate1 f (f x)
```

Questa definizione è lineare ($O(n)$) pur non essendo circolare. Ogni passo produce un elemento applicando la funzione all'argomento corrente, senza accumulare operazioni sospese (come catene di `map`) che invece renderebbero la computazione quadratica.

Produttività

La “regola d'oro” per gli stream è garantire la **produttività**: il processo di generazione deve

consumare meno input di quanto output produce.

- esempio non produttivo: `incstnts = (head incstnts) : (tail incstnts)`.

la definizione richiede di consumare la testa della lista prima ancora di averla generata, portando a una mancata terminazione (il processo “si mangia la coda”)

3.6 Liste infinite come limiti

Una lista infinita può essere vista come il **limite delle sue approssimazioni finite**.

Per gestire l'infinito e il calcolo parziale, Haskell usa il simbolo $\perp$ (bottom, anche usato per la contraddizione in logica), che rappresenta una computazione che non termina o che ha dato errore (corrisponde ad `undefined`).

L'esecuzione di un programma può essere vista come un **accumulo di informazioni**:

- un valore non ancora calcolato ($\perp$) non ha informazioni
- man mano che il programma avanza, $\perp$ viene sostituito da valori reali (es: $1 : \perp$, poi $1 : 2 : \perp$)

Possiamo quindi definire una relazione di ordine parziale $\sqsubseteq$, l'**ordine di approssimazione**.

Diciamo che $x \sqsubseteq y$ (x approssima y) se y contiene almeno tutte le informazioni di x .

Per esempio, una lista infinita `[1..]` può essere vista come il limite di una sequenza di approssimazioni che diventano sempre più precise:

$$\perp \sqsubseteq 1 : \perp \sqsubseteq 1 : 2 : \perp \sqsubseteq 1 : 2 : 3 : \perp \dots$$

Il limite di questa catena è la lista infinita completa.

(Invece, una lista come $\perp, 1 : \perp, 2 : 1 : \perp, 3 : 2 : 1 : \perp, \dots$ non converge a nessun limite: non si stabilizza a nessun valore e offre informazioni contraddittorie).

3.6.1 Domini, ordine e limiti

Vediamo nel dettaglio come Haskell misura l'informazione prodotta da un programma.

3.6.1.1 Tipi semplici

Per quanto riguarda i tipi semplici, l'informazione è un “tutto o niente”: o il valore è calcolato, o non lo è.

La relazione $\sqsubseteq$ per i tipi semplici è *piatta* (**flat domain**):

$$x \sqsubseteq y \text{ sse } x = \perp \text{ oppure } x = y$$

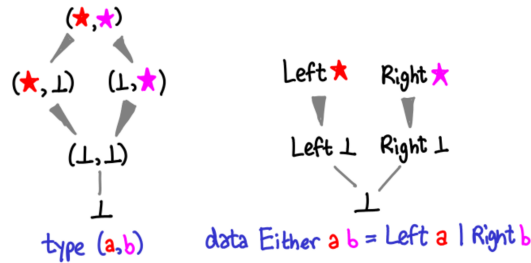
Si ha quindi per esempio $\perp \sqsubseteq \text{True}$, ma non ci sono relazioni tra `True` e `False`.

3.6.1.2 Costruttori di tipo

L'ordine si estende naturalmente ai tipi composti. L'idea è che una struttura è “più definita” di un'altra se i suoi componenti lo sono.

- *Coppie*: $(x, y) \sqsubseteq (x', y') \iff x \sqsubseteq x' \wedge y \sqsubseteq y'$
- *Either*:

- $\perp \sqsubseteq \text{Left } \perp, \text{Right } \perp$
- $\text{Left } x \sqsubseteq \text{Left } x' \iff x \sqsubseteq x'$
- $\text{Right } x \sqsubseteq \text{Right } x' \iff x \sqsubseteq x'$

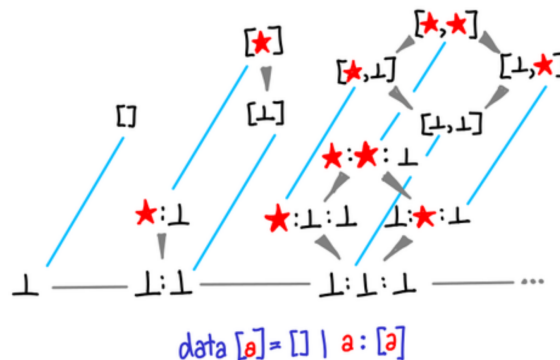


3.6.1.3 Liste e funzioni

Per le liste si ha:

- $\perp \sqsubseteq xs$
- $[] \sqsubseteq xs \iff xs = [] \wedge (x : xs) \not\sqsubseteq []$ (una lista vuota approssima un'altra lista solo se anche essa è vuota; una lista non vuota non può mai approssimare una lista vuota)
- $x : xs \sqsubseteq y : ys \iff x \sqsubseteq y \wedge xs \sqsubseteq ys$

Per esempio, $[1, \perp, 3]$ e $1 : 2 : \perp$ sono approssimazioni di $[1, 2, 3]$.



Invece, una funzione f ne approssima un'altra g se per ogni possibile input x , l'output di f approssima quello di g :

$$\text{date } f, g : a \rightarrow b \text{ si ha } f \sqsubseteq_{a \rightarrow b} g \iff \forall x. fx \sqsubseteq_b gx$$

3.6.1.4 Ordine e limiti

Def. 6: Complete Partial Order

Se abbiamo una catena di approssimazioni $x_1 \sqsubseteq x_2 \sqsubseteq x_3 \dots$, questa tenderà ad un limite $\sqcup x = \lim_{n \rightarrow \infty} x_n$ che soddisfa le seguenti condizioni:

- per ogni n , $x_n \sqsubseteq \sqcup x$ (il limite è un maggiorante)
- se per ogni n , $x_n \sqsubseteq y$, allora $\sqcup x \sqsubseteq y$ (il limite è il supremum, il minimo dei maggioranti (least upper bound))

(simile al lemma di Zorn ma più forte)

Un ordinamento che soddisfa queste proprietà si definisce **Complete Partial Order**

I tipi per domini di funzioni computabili (e quindi eseguibili da una macchina) sono solitamente CPO.

Def. 7: Funzioni computabili

Una funzione computabile gode di due proprietà:

- (1) **monotonia**: $x \sqsubseteq y \implies fx \sqsubseteq fy$
- (2) **continuità**: $f(\sqcup x) = \sqcup fx$

Approfondimento: computabilità logica vs. semantica

È interessante comparare la definizione di funzione computabile fornita dalla teoria dei domini a quella classica della logica matematica (funzioni μ -ricorsive).

Mentre la definizione logica è di tipo costruttivo - identifica la classe delle funzioni calcolabili come la minima classe ottenuta partendo da funzioni base (zero, successore, proiezioni) e applicando operatori di chiusura (composizione, minimalizzazione) -, la definizione basata sui CPO è di tipo semantico. Essa stabilisce i vincoli necessari affinché una funzione possa operare su informazioni parziali o infinite:

- La *monotonia* garantisce che la computazione sia un processo ad accumulo di informazione: se l'input diventa più definito ($x \sqsubseteq y$), l'output non può diminuire o cambiare l'informazione già prodotta ($fx \sqsubseteq fy$)
- La *continuità* formalizza la natura finitaria della calcolabilità: il valore di una funzione su un input infinito (limite) deve essere determinato esclusivamente da ciò che la funzione produce sulle approssimazioni finite di quell'input ($f(\sqcup x) = \sqcup fx$)

In questo contesto, la parzialità delle funzioni logiche viene “internalizzata” nel dominio tramite l'elemento $\perp$ (bottom), che rappresenta il livello minimo di informazione o una computazione non terminata. Questo permette di trattare la non-terminazione non come un'assenza di valore, ma come un valore matematico ben definito su cui è possibile applicare i teoremi di punto fisso per dare un senso alla ricorsione.

Alcuni esempi di funzioni non computabili sono:

- $f(x) = \begin{cases} 0 & x = \perp \\ 1 & x \neq \perp \end{cases}$
 - non è computabile perché non è monotona: se l'input cresce da $\perp$ a un valore definito x ($\perp \sqsubseteq x$), la monotonia richiederebbe che anche i rispettivi output mantengano l'ordine ($f(\perp) \sqsubseteq f(x)$); tuttavia, $0 \not\sqsubseteq 1$
- $f(xs) = \begin{cases} \perp & xs \text{ è finita o parziale} \\ 1 & xs \text{ è infinita} \end{cases}$
 - non è computabile perché non è continua: $\sqcup g x_n = \perp \neq 1 = g(\sqcup x_n)$

3.7 Teoremi di punto fisso

Nella semantica denotazionale, l'esecuzione di un programma ricorsivo è vista come la ricerca di un "punto fisso" di una funzione. Per formalizzare questo concetto, dobbiamo estendere la nozione di CPO a strutture algebriche più ricche.

Def. 8: Reticolo Completo

Un **reticolo completo** (Complete Lattice) $(L, \sqsubseteq)$ è un insieme parzialmente ordinato in cui ogni sottoinsieme $A \subseteq L$ possiede sia un estremo inferiore che un estremo superiore:

- Estremo superiore (Supremum): $\sqcup A = \min\{x \mid \forall a \in A. a \sqsubseteq x\}$ (minimo insieme di maggioranti)
- Estremo inferiore (Infimum): $\sqcap A = \max\{x \mid \forall a \in A. x \sqsubseteq a\}$ (massimo insieme di minoranti)

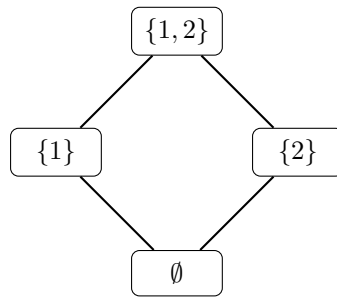


Figura 3.1: Reticolo completo dell'insieme delle parti $\mathcal{P}(\{1, 2\})$ ordinato per inclusione ($\subseteq$); l'estremo inferiore $\perp$ è l'insieme vuoto $\emptyset$, mentre l'estremo superiore $\top$ è l'insieme completo

Avere un reticolo completo ci permette di enunciare due teoremi fondamentali che garantiscono l'esistenza (e la calcolabilità) dei punti fissi per le funzioni su questi domini (un valore x è un *punto fisso* per una funzione f se $f(x) = x$).

Def. 9: Teorema di Knaster-Tarski

In un reticolo completo $(L, \sqsubseteq)$, **ogni funzione monotona** ammette un massimo e un minimo punto fisso.

Il teorema di Knaster-Tarski assicura l'esistenza del punto fisso, ma non dice come trovarlo. Il teorema di Kleene ci fornisce invece un metodo costruttivo.

Def. 10: Teorema di Kleene (del minimo punto fisso)

Se una funzione f è **continua**, il suo minimo punto fisso (Least Fixed Point, LFP) può essere costruito iterativamente partendo dal livello minimo di informazione ($\perp$).

Esso è definito come il limite delle applicazioni ripetute di f :

$$\text{LFP}(f) = f^\omega(\perp) = \lim_{n \rightarrow \infty} f^n(\perp) = \sqcup \{\perp, f(\perp), f(f(\perp)), \dots\}$$

proof

Sappiamo che la sequenza $\perp \sqsubseteq f(\perp) \sqsubseteq f(f(\perp)) \dots$ è una catena monotona crescente e che quindi (poiché ci troviamo in un CPO) ha un estremo superiore.

Possiamo dimostrare che questo estremo superiore f^ω è un punto fisso sfruttando la continuità:

$$\begin{aligned}
 f^\omega &= \sup\{f^i(\perp) \mid i < \omega\} && \text{(per definizione di } f^\omega\text{)} \\
 &= \sup\{f(f^i(\perp)) \mid i < \omega\} && \text{(spostando l'indice di 1 passo)} \\
 &= f(\sup\{f^i(\perp) \mid i < \omega\}) && \text{(per la continuità di } f\text{)} \\
 &= f(f^\omega) && \text{(sostituendo la definizione di } f^\omega\text{)}
 \end{aligned}$$

Poiché $f(f^\omega) = f^\omega$, f^ω è per definizione un punto fisso di f .

Significato computazionale dei teoremi di punto fisso

Quando scriviamo una definizione ricorsiva come `ones = 1 : ones`, stiamo implicitamente definendo un'equazione: $x = f(x)$, dove la funzione è $f(x) = 1 : x$ - stiamo quindi cercando il punto fisso di f .

Il Teorema di Kleene ci spiega esattamente cosa fa la valutazione lazy: il computer costruisce la lista infinita partendo dal non-calcolato ($\perp$) e applicando la funzione in modo continuo:

- (1) $f^0(\perp) = \perp$ (nessuna informazione)
- (2) $f^1(\perp) = 1 : \perp$ (primo elemento noto)
- (3) $f^2(\perp) = 1 : 1 : \perp$ (secondo elemento noto)

Il limite di questo processo all'infinito (f^ω) è proprio la nostra lista infinita. La continuità garantisce che per estrarre una quantità di informazione finita dall'output (es. `take 2 ones`), il computer debba compiere solo un numero finito di passi, senza bloccarsi cercando di valutare l'intera struttura infinita.

3.8 Induzione e co-induzione

Nella teoria dei tipi e nella programmazione funzionale, le strutture dati possono essere definite in due modi duali: tramite induzione (costruendo valori) o tramite co-induzione (osservando valori). Queste due visioni corrispondono alla ricerca di minimi e massimi punti fissi.

Def. 11: Insiemi (Algebre) Induttivi

Un insieme **induttivo** A viene definito “partendo dal basso” (tipicamente dall'insieme vuoto), definendo le regole (costruttori) che permettono di creare valori di tipo A .

- A è il **minimo** insieme chiuso rispetto alle operazioni di costruzione (rappresentato da un assioma di minimalità, es. “nient'altro è un numero naturale”);
- L'insieme A rappresenta il **minimo punto fisso** di un'operazione di costruzione;

- Gli elementi di A sono il risultato di un'applicazione **finita** dei costruttori, a partire da costruttori costanti.
- Da un punto di vista logico, ad A è associato un **principio di induzione** per provare proposizioni su A .
- Da un punto di vista informatico (minima algebra / oggetto iniziale in Teoria delle Categorie), ad A è associato un principio di **ricorsione e/o iterazione** (come `fold`) per definire funzioni che “consumano” il tipo: $f: A \rightarrow B$.

Def. 12: Insiemi (Co-algebre) Co-induttivi

Un insieme **co-induttivo** A viene definito “partendo dall’alto” (ad esempio da un universo U), definendo le regole (distruttori) che permettono di riconoscere i valori che *non* appartengono ad A .

- A è il *massimo* insieme chiuso rispetto alle operazioni di eliminazione.
- L'insieme A rappresenta il **massimo punto fisso** di un'operazione di eliminazione.
- Gli elementi scartati ($U \setminus A$) vengono riconosciuti dopo un numero finito di applicazioni dei distruttori, ma per gli elementi di A questo processo può continuare all'infinito (es. liste infinite o stream).
- Da un punto di vista logico, ad A è associato un **principio di co-induzione** (come la bisimulazione), usato solitamente per provare *equivalenze* su A .
- Da un punto di vista informatico (massima algebra / oggetto finale in Teoria delle Categorie), ad A è associato un principio di **co-ricorsione e/o co-iterazione** (come `unfold` o `iterate`) per definire funzioni che “generano” o costruiscono oggetti di tipo A : $f: B \rightarrow A$.

3.8.1 Insiemi induttivi (e relativi funzionali)

Approfondimento: lemma di Lambek

Se consideriamo l'algebra induttiva come un *oggetto iniziale* nella Teoria delle Categorie, il principio di ricorsione in programmazione è l'incarnazione computazionale dell'**omomorfismo unico** definito dal Lemma di Lambek (vedi appunti del corso “Linguaggi di programmazione”)

Definire una funzione ricorsiva $f: A \rightarrow B$ su un tipo induttivo A significa costruire l'unico omomorfismo unico che mappa i costruttori di A nelle operazioni di un'altra algebra B .

Ogni tipo induttivo ha il proprio operatore fondamentale (un costrutto di ordine superiore) che si occupa di “costruire” questo omomorfismo:

- **Tipi “non ricorsivi” (Booleani, Either):** anche se raramente definiti come tipi ricorsivi, possiedono un principio di ricorsione. Per i booleani è semplicemente l'`if-then-else`, mentre per il tipo `Either` è l'applicazione di due funzioni g e h a seconda del costruttore (`Left` o `Right`). Il principio di induzione associato è l'analisi per casi.
- **Numeri naturali ($\mathbb{N}$):** l'operatore matematico che costruisce l'omomorfismo è ρ (**iteratore**). In Haskell, esso prende la forma della funzione `iter`:

```

1 iter f b 0 = b
2 iter f b n = f (iter f b (n-1))

```

Questo comportamento ricalca esattamente la semantica dei **Numerali di Church**.

Per ottenere una potenza espressiva maggiore, si passa allo schema della *ricorsione primitiva* (l'operatore `rec`), in cui la funzione f riceve in input anche il contatore ($n - 1$):

```

1 rec f b 0 = b
2 rec f b n = f (n-1) (rec f b (n-1))

```

(`rec` permette di definire facilmente funzioni il cui risultato dipende sia dal risultato del passo ricorsione precedente, che dal “contatore”; per esempio, il fattoriale:

```

1      -- f prende il contatore e il risultato della ricorsione
2 factorial n = rec (\cont res -> (cont + 1) * res) 1 n
3      -- fattoriale di n = n (ovvero contatore+1) * fattoriale (n-1) (passo prec
      )

```

)

- **Liste finite:** sulle liste l'operazione che costruisce l'omomorfismo “sostituendo” i costruttori della lista con operazioni su un altro tipo è la `foldr`. Rappresenta il principio di iterazione/ricorsione generalizzato per le liste:

```

1 iter f b [] = b
2 iter f b (x:xs) = f x (iter f b xs)  -- corrisponde a foldr f b xs

```

3.8.2 Insiemi coinduttivi (e relativi funzionali)

L'insieme coinduttivo che origina dai costruttori dei numeri naturali è formato dai naturali, più un **naturale infinito non-standard** $\omega = S^\infty$, ottenuto da infinite applicazioni del costruttore S .

L'insieme coinduttivo più interessante è quello delle liste infinite (o **Stream**). Non ha senso definire funzioni per ricorsione su di esse, ma si possono sempre definire funzioni che creano Stream di cui, in tempo finito, si può accedere ad una porzione finita.

Il principio di ricorsione sulle Stream si può definire tramite due funzionali:

- `iterate`, che permette di definire una funzione `f :: a -> Stream b`
data una funzione `f :: a -> b` ed un valore `x :: b`, permette di costruire uno stream in questo modo:

```

1 iterate f x = x : iterate f (f x)

```

(crea quindi lo stream `x : f x : f (f x) : ...`)

Conviene però avere un principio di ricorsione più generale: la cosiddetta **co-ricorsione**, ottenibile con il funzionale `unfold`.

```

1 unfold :: (b -> (a, b)) -> b -> Stream a
2 unfold f y = x : unfold f y' where (x, y') = f y

```

Il funzionale `unfold` può essere visto come un **generatore infinito** regolato da una macchina a stati. Consideriamo la firma del suo tipo:

$$\text{unfold} :: (b \rightarrow (a, b)) \rightarrow b \rightarrow \text{Stream } a$$

- Il tipo b (**seme o stato interno**) rappresenta l'informazione corrente necessaria e sufficiente per calcolare il prossimo elemento dello stream.
- La funzione $f :: b \rightarrow (a, b)$ (**generatore**) “apre” (da cui *unfold*) lo stato attuale y per produrre una coppia (x, y') , in cui:
 - x (di tipo a) è il **valore** effettivo che viene emesso in output ed entra a far parte dello Stream
 - y' (di tipo b) è il **nuovo seme** che verrà riutilizzato come argomento al passo successivo della computazione

A ogni iterazione, il funzionale prende lo stato corrente, produce un elemento visibile all'esterno e aggiorna lo stato interno per la computazione futura, “srotolando” il seme iniziale all'infinito.

Produttività

Nelle strutture dati infinite, il concetto classico di *terminazione* perde di significato (lo stream, per definizione, non deve mai finire). Al suo posto, si introduce il criterio di **produttività**:

Una funzione co-ricorsiva si dice **produttiva** se è in grado di valutare una qualsiasi porzione finita del suo output (ad esempio l'elemento di testa) in un **tempo finito**.

Nel codice di `unfold`, la produttività è garantita dal costruttore di lista `( : )`. Poiché il costruttore si trova all'esterno della chiamata co-ricorsiva `( x : unfold f y' )`, Haskell può istanziare immediatamente la testa `x` e congelare la computazione del resto dello stream `( unfold f y' )` sotto forma di *thunk*, finché non sarà esplicitamente richiesta da una successiva operazione di accesso.

Possiamo ridefinire il funzionale `iterate` precedentemente analizzato come un caso particolare di co-ricorsione in cui lo stato interno e il tipo dell'elemento emesso coincidono ($a = b$):

```

1 iterate :: (a -> a) -> a -> Stream a
2 iterate f x = unfold (\s -> (s, f s)) x

```

In questo caso, il generatore prende lo stato `s`, lo restituisce come elemento corrente dello stream e applica `f s` per determinare il nuovo seme per il passo successivo.

Se invece vogliamo considerare l'algebra che contiene la lista vuota, allora la massima co-algebra contiene anche le liste finite. In tal caso, possiamo definire il funzionale `unfoldLS`, che costruisce anche liste finite:

```

1 unfold :: (b -> Bool) -> (b -> (a, b)) -> b -> [a]
2 unfold p f y = if p y then []
3               else x : unfold p f y' where (x, y') = f y

```

3.8.3 Bisimulazione

Come per gli insiemi induttivi, abbiamo bisogno di un modo per derivare equivalenze su insiemi co-induttivi. La controparte dell'induzione che ci permette di farlo è la **bisimulazione**.

L'idea alla base della bisimulazione è osservare se due processi (o liste) “si comportano” allo stesso modo per ogni osservazione finita che possiamo farne.

Si procede in questo modo:

- (1) si definisce una relazione R tra due liste
- (2) se si dimostra che per ogni coppia $(xs, ys) \in R$, le loro teste sono uguali e le loro code sono ancora in R , allora le liste si dicono **bisimili** ($\approx$)

Formalmente, si definisce una bisimulazione R tra coppie di liste come segue:

$$R \text{ } xs \text{ } ys \implies xs = ys = \perp \text{ oppure } xs = ys = [] \\ \text{oppure } \exists v, vs, ws \text{ t.c. } xs = v : vs \wedge ys = v : ws \wedge vs R ws$$

Definiamo quindi $xs \approx ys$ se esiste una bisimulazione R tale che $xs R ys$, il che equivale a scegliere la **massima equivalenza**, definita come massimo punto fisso.

Prova per bisimulazione

Possiamo dimostrare per bisimulazione che:

```
1 map f (iterate f x) = iterate f (f x)
```

Ci basta dimostrare che la relazione R che contiene:

$$\{ \text{map } f \text{ (iterate } f \text{ } x), \text{ iterate } f \text{ (f } x) \mid f, x \text{ opportuni} \}$$

è una bisimulazione.

Facendo ricorso all'algebra dei programmi, possiamo “svolgere” le due espressioni:

```
1 map f (iterate f x) = map f (x : iterate f (f x))
2                   = f x : map f (iterate f (f x))
```

e anche

```
1 iterate f (f x) = f x : iterate f (f (f x))
```

Entrambe producono la stessa testa $(f \text{ } x)$, e le code sono in R (in cui la nuova funzione applicata sarà $f.f$ invece di f).

3.8.4 Dimostrazione per approssimazione

L'alternativa alla co-induzione consiste nell'effettuare una dimostrazione per induzione sulle approssimazioni finite di una lista. Per farlo, si internalizza in Haskell la funzione `approx`, strettamente imparentata con `take`:

```

1 approx 0 xs = undefined
2 approx n [] = []
3 approx n (x:xs) = x : approx (n-1) xs

```

La proprietà fondamentale di `approx` è che il limite delle approssimazioni di una lista è la lista stessa:

$$\sqcup_n \text{approx } n \text{ } xs = xs$$

Di conseguenza, per dimostrare l'uguaglianza tra due liste xs e ys , è sufficiente mostrare che tutte le loro approssimazioni finite siano uguali per ogni n :

$$\forall n, \text{approx } n \text{ } xs = \text{approx } n \text{ } ys \implies xs = ys$$

Esempio

Proviamo a dimostrare nuovamente la proprietà:

$$\text{approx } n \text{ } (\text{map } f \text{ } (\text{iterate } f \text{ } x)) = \text{approx } n \text{ } (\text{iterate } f \text{ } (f \text{ } x))$$

- **caso base:** per $n = 0$ la proprietà è banalmente vera perché `approx 0 xs` restituisce sempre `⊥` (undefined).
- **caso induttivo:** si procede riducendo le due espressioni separatamente tramite program calculation e applicando l'ipotesi induttiva:

$$\begin{aligned}
& \text{approx } (n+1) \text{ } (\text{iterate } f \text{ } (f \text{ } x)) \\
&= \{\text{def. di iterate}\} \\
& \text{approx } (n+1) \text{ } (f \text{ } x : \text{iterate } f \text{ } (f \text{ } (f \text{ } x))) \\
&= \{\text{def. di approx}\} \\
& f \text{ } x : \text{approx } n \text{ } (\text{iterate } f \text{ } (f \text{ } (f \text{ } x))) \\
&= \{\text{ipotesi induttiva}\} \\
& f \text{ } x : \text{approx } n \text{ } (\text{map } f \text{ } (\text{iterate } f \text{ } (f \text{ } x))) \\
&= \{\text{def. di approx al contrario}\} \\
& \text{approx } (n+1) \text{ } (f \text{ } x : \text{map } f \text{ } (\text{iterate } f \text{ } (f \text{ } x))) \\
&= \{\text{def. di map al contrario}\} \\
& \text{approx } (n+1) \text{ } (\text{map } f \text{ } (x : \text{iterate } f \text{ } (f \text{ } x))) \\
&= \{\text{def. di iterate al contrario}\} \\
& \text{approx } (n+1) \text{ } (\text{map } f \text{ } (\text{iterate } f \text{ } x))
\end{aligned}$$

3.9 Funtori e Applicativi

Concetti principali

Sia i Funtori che gli Applicativi servono a gestire computazioni all'interno di un **contesto**, ma con "poteri" diversi:

- **Functors:** permettono di applicare una funzione "pura" a un valore nel contesto - modifi-

cano il contenuto ma non la struttura del contesto;

- **Applicative:** permettono di applicare funzioni che sono esse stesse dentro un contesto a valori nel contesto - sono necessari per gestire funzioni con più argomenti;

classe	operatore	scopo
Functor	<code>fmap :: (a -> b) -> t a -> t b</code>	trasformazione (1 argomento)
Applicative	<code><*> :: t (a -> b) -> t a -> t b</code>	combinazione (n argomenti)

3.9.1 Funtori

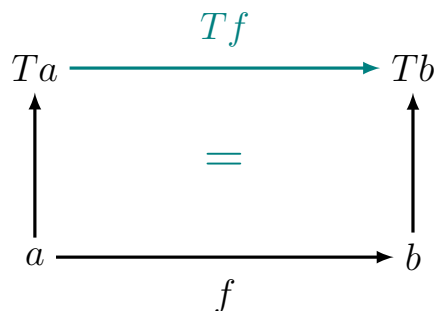
L'idea alla base di un **funtore** è la generalizzazione del funzionale `map`, tipicamente utilizzato per le liste, a tutti i possibili costruttori di tipo.

Se si ha:

- una funzione $f :: a \rightarrow b$
- un costruttore di tipo T che trasforma un tipo a in un tipo ta (come per esempio `[a]` oppure `Maybe a`)

Si può ottenere in modo canonico una funzione

$$Tf :: ta \rightarrow tb$$



Nel mondo categoriale, il concetto di “funtore” descrive un passaggio tra due categorie: dati due oggetti (tipi come a o b), un funtore agisce come un costruttore di tipo (T) - prende un oggetto a nella categoria C e lo trasforma in un oggetto ta nella categoria D .

- il punto importante è che un funtore non mappa solo i tipi, ma anche le relazioni tra essi: se esiste una funzione $f : a \rightarrow b$, il funtore deve fornire un modo per ottenere una funzione corrispondente tf che operi sui tipi $ta \rightarrow tb$
- un funtore è quindi una **mappa che preserva la struttura**

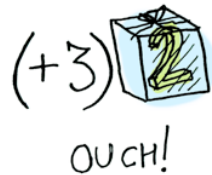
La classe `Functor` richiede che sia definita una funzione `fmap`, che rende commutativo il passaggio dai tipi base ai tipi “in scatolati”

```
1 class Functor t where
2   fmap :: (a -> b) -> t a -> t b
```

- il tipo di `fmap` è parametrico, oltre che nei tipi `a` e `b`, anche rispetto a un **costruttore di tipo** `t` (si vede che `t` deve essere un costruttore di tipo perché si applica ad altri tipi)

`fmap` prende quindi una funzione $a \rightarrow b$ e un valore “in scatolato” $t\ a$, e :

- “spacchetta” il valore
- applica la funzione
- “impacchetta” il risultato



Esempi

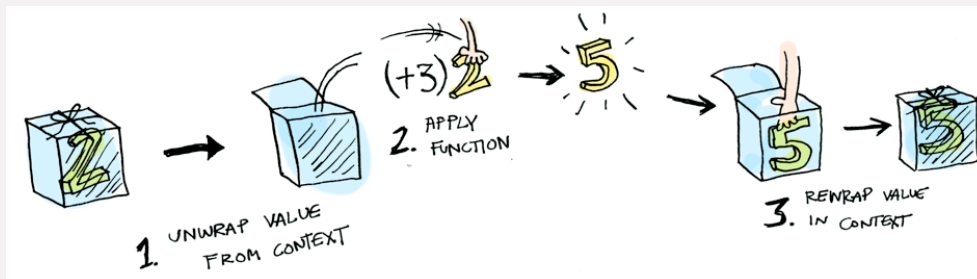
Alcuni esempi noti di istanze di funtori sono:

- le liste

```
1 instance Functor [] where
2   fmap = map
```

- il tipo Maybe

```
1 instance Functor Maybe where
2   fmap f Nothing = Nothing
3   fmap f (Just x) = Just (f x)
```



Possiamo definire i funtori anche per esempio sugli alberi binari:

```
1 instance Functor BinTree where
2   fmap f (Leaf x) = Leaf (f x)
3   fmap f (Branch l r) = Branch (fmap f l) (fmap f r)
```


Definire un tipo come istanza di `Functor` permette di scrivere codice generico che funziona su diverse strutture dato.

Per esempio, si può definire una funzione `inc` che incrementa i valori all'interno di qualsiasi funtore:

```
1 inc :: Functor t => t Int -> t Int
2 inc = fmap (+1)
```

3.9.1.1 Funzioni come funtori

Un'estensione interessante del concetto di funtore si ha quando consideriamo le **funzioni stesse** come istanze della classe `Functor`.

In questo caso, il costruttore di tipo è l'operatore freccia parzialmente applicato, ovvero `((->) r)`. Questo costruttore prende un tipo a e lo trasforma nel tipo di una funzione che si aspetta un input di tipo r , cioè $r \rightarrow a$.

Se proviamo a sostituire il costruttore `t` con `((->) r)` nella firma di `fmap`, otteniamo:

```
fmap :: (a -> b) -> ((->) r a) -> ((->) r b)
```

Che riscritta in forma infissa diventa:

```
fmap :: (a -> b) -> (r -> a) -> (r -> b)
```

Questa firma coincide con quella dell'operatore di **composizione di funzioni** `(.)`. Di conseguenza, l'istanza si definisce semplicemente così:

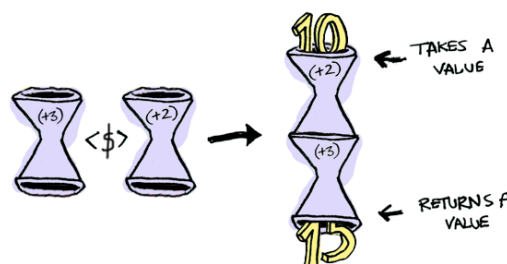
```
1 instance Functor ((->) r) where
2     fmap f g = f . g
3     -- o in forma contratta: fmap = (.)
```

Intuizione

L'idea di "contenitore" qui sfuma in quella di **contesto computazionale**:

- un valore di tipo $r \rightarrow a$ può essere visto come una scatola che non contiene ancora un valore di tipo a , ma sa come produrlo non appena le viene fornito un input di tipo r
- applicare `fmap f g` significa post-elaborare il risultato della computazione: non appena `g` produrrà il suo valore di tipo a , questo verrà immediatamente passato a `f` per ottenere un tipo b

Grazie a questa proprietà, mappare una funzione sopra un'altra significa semplicemente creare una pipeline logica in cui il flusso dei dati attraversa le funzioni in sequenza.



3.9.1.2 Functor laws

Affinché un funtore sia definito correttamente, è necessario che rispetti le due functor laws:

- (1) `fmap id = id`
- (2) `fmap (f . g) = fmap f . fmap g`

Nota bene: queste leggi non possono essere verificate dal type-checker, ma sono responsabilità del programmatore Haskell !

Funtori come omomorfismi

Possiamo notare, osservando le *functor laws*, che un funtore è effettivamente un **omomorfismo tra categorie** (un omomorfismo “classico” preserva l’operazione di gruppo, mentre un funtore preserva la “struttura” di una categoria, ovvero gli oggetti - i tipi -, e i morfismi - le funzioni.)

3.9.1.3 Notazione infissa

Il modulo `Data.Functor` definisce anche una notazione alternativa per `fmap`: la sua versione infissa, `<$>`.

```
1 (<$>) :: Functor t => (a -> b) -> t a -> t b
2 f <$> x = fmap f x
3
4 -- (+1) <$> [1, 2, 3] -> [2, 3, 4]
```

3.9.1.4 Lifting

Come abbiamo visto all’inizio di questa sezione, un modo di vedere `fmap` è attraverso il *currying* del suo tipo. Cambiando l’ordine concettuale delle parentesi nella segnatura:

```
fmap :: (a -> b) -> (t a -> t b)
```

In quest’ottica, `fmap` prende una funzione normale $a \rightarrow b$ e la **solleva** nel mondo dei contesti, trasformandola in una funzione tra strutture $ta \rightarrow tb$. Per questo motivo, a volte in Haskell si trova la funzione storica `liftM` (o simili), che concettualmente fa la stessa cosa.

3.9.1.5 Il limite dei Funtori: funzioni multi-argomento

Cosa succede se proviamo a mappare una funzione che accetta due o più argomenti (come l’addizione `(+) :: Int -> Int -> Int`) su un funtore?

Se valutiamo:

```
1 (+) <$> Just 3
```

Ricordando che in Haskell tutte le funzioni sono curryficate, `(+)` applicata a 3 restituisce una funzione parziale `(3+)`. Di conseguenza, il tipo del risultato sarà:

```
1 Just (3+) :: Maybe (Int -> Int)
```

Abbiamo ottenuto una funzione inscatolata nel contesto. Il limite di `Functor` sta nel fatto che con `fmap` non abbiamo alcun modo di applicare questa funzione `Maybe (Int -> Int)` a un altro valore anch'esso dentro un `Maybe` (ad esempio `Just 5`).

3.9.2 Applicativi

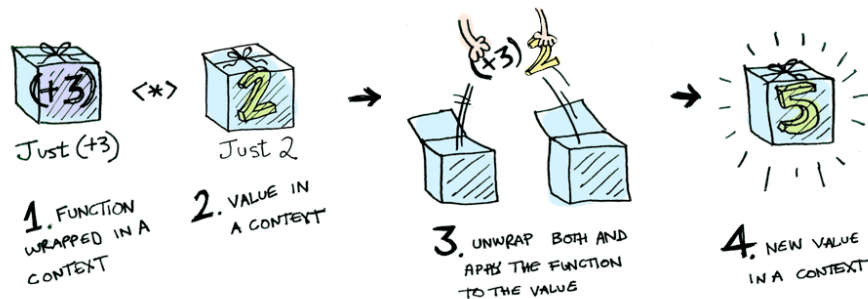
Un normale funtore permette di usare `fmap` con un solo argomento, ma è naturale voler generalizzare: si vuole considerare una forma astratta di applicazione, che generalizzi l'usuale applicazione di funzioni. Per esempio, si vuole poter propagare un fallimento nel caso di `Maybe`, o applicare una funzione n -aria a valori "inscatolati" senza dover istanziare n funtori.

Haskell definisce quindi la classe `Applicative`.

```
1 class Functor t => Applicative t where
2   pure :: a -> t a
3   (<*>) :: t (a -> b) -> t a -> t b
```

Un applicativo estende un funtore, e richiede due operatori fondamentali:

- `pure` : prende un valore "normale" e lo immerge nel contesto (lo "impacchetta")
per esempio, per `Maybe`, `pure x` è `Just x`
- `<*>` (*splat* o *apply*): prende una funzione "impacchettata" in un contesto (`t (a -> b)`) e un valore anch'esso in un contesto (`t a`), "scarta" concettualmente entrambi i contesti, applica la funzione al valore (gestendo gli effetti del contesto), produce il risultato e lo "incarta" nel contesto (`t b`)



Possiamo istanziare `Maybe` anche come applicativo:

```
1 instance Applicative Maybe where
2   pure = Just
3   Nothing <*> _ = Nothing -- se uno dei due è Nothing, entrambi sono Nothing
4
5   -- potremmo fare
6   -- Just f <*> Just x = Just (f x) ma abbiamo già fmap dei Funtori !
7   -- se "spacchettiamo" f e non x, otteniamo esattamente la segnatura
8   -- dei funtori (a->b) -> f a e possiamo fare:
9
10  (Just f) <*> mx = fmap f mx
11  -- <*> prende quindi una funzione e un valore in un contesto
12  -- per es. Just (+3) <*> Just 2 = Just 5
```

Combinare `<$>` e `<*>` : stile applicativo

```
1 (*) <$> (Just 8) <*> (Just 2)
2 -- valuta a Just 16 !
```

Questo esempio mostra come combinare `fmap (<$>)` e `<*>` :

- l'operatore `<$>` (`fmap`) prende la funzione binaria “pura” `(*)` e la mappa sul primo argomento contestualizzato `(Just 8)` ; questo produce una funzione parzialmente applicata dentro il contesto: `Just (8 *)` .
- poiché ora la funzione è “impacchettata” nel contesto `Maybe` , non possiamo più usare un normale `fmap` per passarle il secondo argomento
- usiamo quindi `<*>` , che prende la funzione nel contesto `Just (8 *)` , la applica al valore nel contesto `(Just 2)` e propaga l'effetto, restituendo `Just 16`

Questo pattern si estende a funzioni con quanti argomenti si desidera, alternando sempre un `<$>` iniziale e una catena di `<*>` :

```
1 f <$> arg1 <*> arg2 <*> arg3 <*> ... <*> argN
```

Teoricamente, potremmo anche scrivere `pure (*) <*> Just 8 <*> Just 2` .

Tipicamente, si scriverebbe in quel modo per mostrare didatticamente come `<*>` lavori sempre su due strutture dello stesso tipo. Nel codice reale, tuttavia, si preferisce iniziare con `<$>` grazie a una legge di equivalenza fondamentale degli applicativi:

```
1 pure f <*> x == f <$> x
```

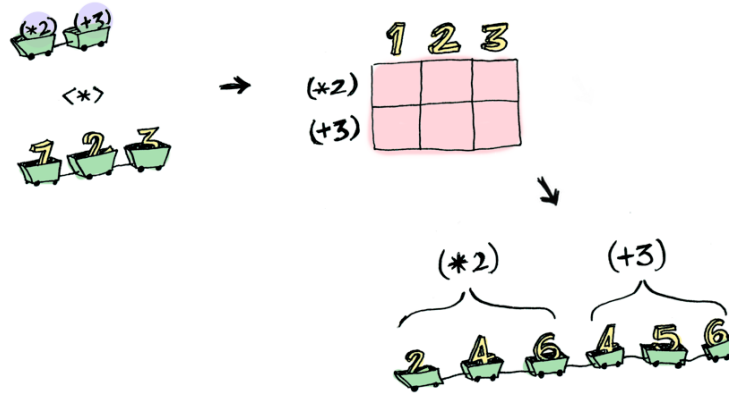
Usare `<$>` all'inizio è la scelta più idiomatica perché evita di dover “impacchettare” artificialmente la funzione per poi doverla subito combinare col primo argomento.

L'operatore `pure` resta comunque indispensabile in due scenari principali:

- inserire valori costanti a metà catena: se una funzione richiede tre argomenti, di cui i primi due contestualizzati e il terzo fisso, usiamo `pure` per iniettarlo senza rompere la catena:
`f <$> magariA <*> magariB <*> pure “valore_fisso”`
- incartare un valore singolo: quando si vuole semplicemente inserire un valore in un contesto senza applicargli alcuna funzione (es. `pure 5 :: [Int]` produce `[5]`).

3.9.2.1 Liste come applicativi

L'istanza del Prelude delle liste non si comporta come una `zipWith` nell'applicare le funzioni presenti in una lista, ma modella un non-determinismo. Una lista non viene vista come una sequenza, ma come un valore che può assumere più risultati possibili contemporaneamente. Quindi, quando si applica una lista di funzioni ad una lista di valori, l'applicativo contiene tutte le combinazioni possibili (prodotto cartesiano).



```

1 instance Applicative [] where
2   -- pure : a -> [a]
3   pure x = [x]
4
5   -- <*> : [a -> b] -> [a] -> [b]
6   fs <*> xs = [f x | f <- fs, x <- xs]
7   -- ^ prodotto cartesiano

```

Se eseguiamo quindi `pure (*) <*> [1,2] <*> [3,4]` (o, in stile applicativo, `(*) <$> [1,2] <*> [3,4]`):

- `pure (*)` crea `[(*)]`
- `[(*)] <*> [1,2]` produce una lista di funzioni `[(1*), (2*)]`
- `[(1*), (2*)] <*> [3,4]` applica entrambe le funzioni ad entrambi i numeri
si ottiene quindi `1*3, 1*4, 2*3, 2*4 → [3, 4, 6, 8]`

A volte si vuole però che le liste si comportino come `zipWith` - per questo, Haskell introduce `ZipList`

```

1 -- per poter definire un'altra istanza di Applicative senza conflitti
2 newtype ZipList a = Z [a]
3
4 instance Applicative ZipList where
5   -- pure : a -> ZipList a
6   pure x = Z (repeat x) -- crea una lista infinita di x
7
8   -- <*> : [a -> b] -> [a] -> [b]
9   Z fs <*> Z xs = Z (zipWith (\f x -> f x) fs xs)

```

- `pure` è `repeat` così che si adatti a qualsiasi lunghezza della lista di valori con cui verrà accoppiata (se restituisse un solo elemento, `zipWith` si fermerebbe subito)
- `<*>` è `zipWith` - applica la funzione in posizione 1 al valore in posizione 1 ecc

3.9.2.2 Esempio: alberi come applicativi

Consideriamo una struttura dati ad albero binario definito sulle foglie:

```

1 data BinTree a = Leaf a | Branch (BinTree a) (BinTree a)
2
3 instance Functor BinTree where
4     fmap f (Leaf x) = Leaf (f x)
5     fmap f (Branch l r) = Branch (fmap f l) (fmap f r)

```

Come per le liste, esistono due modi concettualmente diversi di estendere questa struttura ad un `Applicative`.

1. L'approccio zip-like Il primo modo consiste nell'applicare le funzioni ai nodi corrispondenti per posizione, fondendo le strutture (come per `ZipList`).

In questo caso, `pure` genera un albero infinito composto dalla stessa foglia, in modo da potersi "adattare" a qualsiasi forma dell'albero a cui verrà applicato.

```

1 instance Applicative BinTree where
2     -- pure crea una foglia che si espande virtualmente all'infinito
3     pure x = Leaf x
4
5     -- <*> applica le funzioni ricorsivamente mantenendo la struttura
6     -- caso base: applico la funzione alla foglia
7     (Leaf f) <*> (Leaf x) = Leaf (f x)
8     -- abbiamo una funzione in una foglia ma un branch a cui applicarla;
9     -- si "duplica" la funzione e si applica sia a dx che a sx
10    (Leaf f) <*> (Branch l r) = Branch (fmap f l) (fmap f r)
11    -- per far combaciare le strutture, "copiamo" la foglia
12    (Branch l r) <*> (Leaf x) = Branch (l <*> Leaf x) (r <*> Leaf x)
13    -- la parte sinistra va a sinistra e quella destra a destra
14    (Branch l r) <*> (Branch a b) = Branch (l <*> a) (r <*> b)

```

2. L'approccio combinatorio Se invece vogliamo che l'albero si comporti in modo "non-deterministico" (come l'istanza standard delle liste), ogni ramo dell'albero delle funzioni deve combinarsi con *tutto* l'albero dei valori. Questo approccio potrebbe essere utile per clonare la struttura sotto una nuova radice comune.

Esempio: Duplicare un albero con nuove radici

Immaginiamo di definire l'applicazione in modo che un albero di funzioni si distribuisca interamente sull'albero dei valori:

```

1 (Leaf f) <*> albero = fmap f albero
2 (Branch l r) <*> albero = Branch (l <*> albero) (r <*> albero)

```

Se ora creiamo un albero contenente delle funzioni parziali, ad esempio

`Branch (Leaf (+2)) (Leaf (+3))`, e lo applichiamo tramite `<*>` ad un albero generico, otteniamo:

```

1 Branch (Leaf (+2)) (Leaf (+3)) <*> albero
2 -- Produce un unico albero avente come radice un Branch:
3 -- il sottoalbero sinistro sarà il vecchio albero con tutti i valori
   modificati da (+2),
4 -- il sottoalbero destro sarà il vecchio albero con tutti i valori modificati
   da (+3).

```

In alternativa, senza riscrivere l'istanza di `Applicative`, lo stesso risultato si può ottenere combinando il costruttore `Branch` con `fmap (<$>)` e `<*>`:

```

1 nuovoAlbero = Branch <$> (fmap (+2) albero) <*> (fmap (+3) albero)

```

3.9.2.3 Applicative laws

Come i funtori, anche gli applicatori hanno una serie di regole da rispettare:

(1) `<*>` **preserva l'identità**

```
pure id <*> x = x
```

se prendiamo la funzione identità, la inseriamo in un contesto e la applichiamo ad un valore `x` già contestualizzato, il risultato deve rimanere `x`

(2) `<*>` **preserva l'applicazione di funzioni**

```
pure (g x) = pure g <*> pure x
```

applicare `g` ad un valore `x` e poi “impacchettare” il risultato dà lo stesso esito che applicare la versione “impacchettata” di `g` alla versione “impacchettata” di `x`

(3) **non conta l'ordine** con cui si fa embedding nel contesto

```
f <*> pure y = pure (\g -> g y) <*> f
```

(4) **composizione di <*>**

```
f <*> (g <*> x) = (pure (.) <*> f <*> g) <*> x
```

(forma di associatività per `<*>`)

la composizione di funzioni `(.)` può essere “lifted” a livello applicativo per garantire che la sequenza delle applicazioni sia coerente

- ovvero, applicare due funzioni una dopo l'altra è identico a comporle prima in un'unica funzione e poi applicarla:

```
f <*> (g <*> x)
```

prima si applica la funzione `g` al valore `x` (ottenendo un risultato nel contesto), e poi si applica `f` a tale risultato - è quindi `f(g(x))`

```
(pure (.) <*> f <*> g) <*> x
```

qui l'operatore di composizione standard `(.)` viene “sollevato” (lifted) nel contesto, permettendo di fondere le due funzioni `f` e `g` dentro il contesto - si crea quindi una funzione composta che sarà applicata a `x`

quindi, si applica `f . g` a `x`, ottenendo anche qui `f(g(x))`

3.10 Monadi Base

Le monadi rappresentano il passo successivo rispetto agli applicativi per la gestione controllata di **side-effects**. Se un applicativo permette di combinare calcoli indipendenti, la monade permette di gestire calcoli in cui il passo successivo **dipende** dal risultato di quello precedente.

3.10.1 Introduzione

Per comprendere perché le monadi siano utili, consideriamo un valutatore di espressioni aritmetiche che supporta costanti e divisioni:

```
1 data Term = Const Int | Div Term Term
2
3 eval :: Term -> Int
4 eval (Const a) = a
5 eval (Div t u) = eval t 'div' eval u
```

Questa implementazione è elegante ma fragile: un'espressione come `Div (Const 1) (Const 0)` farà crashare il programma.

Si può usare il tipo `Maybe Int` per modellare il possibile fallimento. Tuttavia, il codice diventa immediatamente prolisso a causa della propagazione manuale dell'errore:

```
1 eval :: Term -> Maybe Int
2 eval (Const a) = Just a
3 eval (Div t u) = case eval t of
4     Nothing -> Nothing
5     Just n -> case eval u of
6         Nothing -> Nothing
7         Just m -> if m == 0 then Nothing
8                   else Just (n 'div' m)
```

La divisione (il vero senso della funzione) è offuscata dalla gestione dei `Nothing`.

Si potrebbe pensare di usare l'operatore `<*>` degli applicativi per pulire il codice. Tuttavia, gli applicativi sono insufficienti quando la struttura del calcolo dipende dal valore dei risultati intermedi:

- L'operatore `<*>` (`f (a -> b) -> f a -> f b`) richiede che la funzione da applicare sia “pura” e che i contesti (`f`) siano già definiti. In un calcolo applicativo, l'intera sequenza di effetti è determinata a priori: non si può usare il valore estratto da un `Maybe` per decidere quale sarà il prossimo “effetto”.
- Se provassimo a usare una funzione che genera essa stessa un contesto, come `safeDiv :: Int -> Int -> Maybe Int`, l'applicazione risulterebbe in un tipo `Maybe (Maybe Int)`.
 - e.g. `pure safeDiv <*> Just 10 <*> Just 0` produce `Just (Nothing)`

L'applicativo non possiede un meccanismo per “appiattire” questi due livelli di contesto in uno solo.

Nella divisione, il fatto che il secondo argomento sia zero determina se il calcolo successivo deve fallire o meno. In Haskell, questa capacità di far sì che il “guscio” esterno (il contesto) venga generato dal valore contenuto nel passo precedente è l'essenza delle Monadi.

In sintesi: gli Applicativi modellano calcoli indipendenti (i contesti sono paralleli), mentre le Monadi (come ora vedremo) modellano calcoli sequenziali, dove il risultato di un'operazione influenza la creazione della successiva.

In termini di Teoria delle Categorie, l'applicativo gestisce contesti indipendenti (funtori monoidali), mentre per concatenare funzioni del tipo $A \rightarrow M(B)$ (chiamate frecce di Kleisli) abbiamo bisogno di una Monade.

3.10.2 Monadi (shall I be pure or impure?)

Molti aspetti della programmazione (come strutture dati mutabili e input/output) non hanno una rappresentazione efficace nel mondo funzionale, mentre altri potrebbero beneficiare da una programmazione con side-effects (come eccezioni e tracciamento delle computazioni).

Alcuni linguaggi funzionali, come ML, introducono costrutti non funzionali come puntatori, eccezioni, ecc.; Haskell cerca un'integrazione basata su concetti matematici.

La classe `Monad` si può definire tramite gli applicativi in questo modo:

```
1 class Applicative m => Monad m where
2   (>=>) :: m a -> (a -> m b) -> m b -- operatore "bind"
3   (>>)  :: m a -> m b -> m b -- "sequence"/"then"
4   return :: a -> m a -- identico a "pure"
```

- `return` (o `pure`) prende un valore e lo inserisce nel contesto minimo necessario
- `>=` (**bind**) prende un valore “impacchettato” (`m a`), lo “spacchetta”, e lo passa ad una funzione che accetta un valore puro e restituisce un nuovo valore impacchettato (`a -> mb`)
 - in questo modo, la computazione viene sequenzializzata: prima viene valutato `m : M a` e poi il risultato viene passato a `f`
- `>>` è una versione semplificata del bind: mentre `>=` prende il valore della prima computazione e lo passa ad una funzione, `>>` esegue la computazione, ne scarta il valore prodotto e passa direttamente alla seconda computazione (il suo tipo è infatti quello della seconda computazione - in una computazione sequenziale, l'ultima istruzione è quella che determina il valore finale)
 - `m >> f` è equivalente a `m >= _ -> f`
 - serve quindi quando si vogliono solo i side-effects ma non quello che viene ritornato (e.g. operazioni di scrittura)
 - in ogni caso, anche se il valore di tipo `a` viene scartato, il contesto (effetto computazionale) di `m a` viene preservato e combinato con quello di `b`

In questo modo, una funzione `f : a -> b` viene rimpiazzata da `f' : a -> M b`, in cui `M` cattura gli effetti computazionali di `f`.

Valutatore generalizzato

Possiamo quindi scrivere una versione generale del nostro valutatore che accetti qualsiasi monade:

```

1 eval :: Monad m => Term -> m Int
2 eval (Const a) = pure a
3 eval (Div t u) = eval t >>= \a ->
4                  eval u >>= \b ->
5                  pure (a 'div' b)

```

Segue un link utile per comprendere al meglio questi concetti: [\[qui\]](#)

3.10.3 Alcune monadi note

3.10.3.1 Monade identità

La **monade identità** è simile all'elemento neutro di un'algebra. Serve a rappresentare una computazione senza un effetto collaterale, e permette di definire una funzione generale ed usarla senza aggiungere nessun comportamento "speciale".

```

1 instance Monad Identity where
2     return a = a
3     -- (>>=) :: Identity a -> (a -> Identity b) -> Identity b
4     a >>= f = f a

```

3.10.3.2 Liste

Anche le **liste** si possono definire come monadi.

```

1 instance Monad [] where
2     return x = [x]
3     xs >>= f = concat (map f xs)
4     -- xs >>= f = [y | x <- xs, y <- f x]

```

La logica è quella di prendere una lista, applicare ad ogni elemento una funzione che restituisce a sua volta una lista, e poi "appiattire" tutto (`>>=` è analogo alle list comprehension).

3.10.3.3 Monade eccezione

Il tipo `Maybe` può essere trattato come una monade per gestire le eccezioni.

L'operatore `>>=` si comporta in questo modo:

- se il risultato precedente è `Nothing`, questo sarà propagato
- se è `Just x`, `x` viene spaccettato e passato alla funzione successiva

```

1 instance Monad Maybe where
2     return x = Just x
3     Nothing >>= _ = Nothing
4     (Just x) >>= f = f x

```

3.10.4 Monade State Transformer

In un mondo imperativo, una variabile può cambiare valore nel tempo. In Haskell, i dati sono immutabili. Per simulare uno “stato” che cambia, si usa una funzione che prende lo stato corrente e restituisce sia un risultato che un nuovo stato.

Per definire una State Transformer “manualmente”, servono:

- uno stato - rappresenta l'informazione che evolve nel tempo (es. un contatore) - per esempio un intero

```
type State = Int.
```

- uno State Transformer (ST) - un wrapper che racchiude la logica del cambiamento di stato. Per definirlo manualmente, si deve usare un costruttore fittizio `S` perché in Haskell non è possibile definire un sinonimo di tipo `( type )` come istanza di una classe:

```
1 newtype ST a = S (State -> (a, State))
```

A causa di questo costruttore, è utile definire un'operazione di **applicazione** (`app`) che rimuove l'etichetta `S` e applica la funzione interna allo stato fornito.

```
1 app :: ST a -> State -> (a, State)
2 app (S st) x = st x
```

Possiamo simulare effetti collaterali (come incrementare un contatore) definendo funzioni specifiche come `tick`. Tuttavia, senza l'uso delle monadi, dovremmo gestire manualmente il “threading” dello stato (passare il nuovo stato `s'` alla computazione successiva), rendendo il codice ripetitivo.

```
1 -- "side effect" manuale
2 tick :: ST Int -> ST Int
3 tick (S st) = S (\s ->
4     let (a, s') = st s -- esegue la computazione e ottiene il nuovo stato
5     in (a, s' + 1))    -- incrementa manualmente lo stato
```

Definendo `ST` come **monade**, l'operatore `>=` si occupa di “nascondere” il passaggio dello stato aggiornato tra le funzioni.

```
1 instance Monad ST where
2     -- return mette un valore in uno stato neutro
3     return x = S (\s -> (x, s))
4
5     -- (>=) concatena le azioni e fa fluire lo stato s' automaticamente
6     st >= f = S (\s ->
7         let (x, s') = app st s
8         in app (f x) s')
```

In Haskell, le monadi sono strutturate secondo una gerarchia di classi. Una monade è, per definizione, un applicativo “potenziato”. Pertanto, per definire `ST` come monade, dobbiamo obbligatoriamente istanziarla come applicativo, e di conseguenza anche come funtore.

Infatti:

- (1) Functor permette di usare `fmap` per trasformare il risultato di una computazione di stato;

- (2) Applicative permette di gestire computazioni indipendenti in sequenza e fornisce `pure` ;
- (3) Monad introduce l'operatore `>=` (bind), permettendo a una computazione di dipendere dal risultato della precedente;

```

1 instance Functor ST where
2   -- fmap :: (a -> b) -> ST a -> ST b
3   fmap f (S st) = S (\s ->
4     let (x, s') = st s
5     in (f x, s'))

```

```

1 instance Applicative ST where
2   pure x = S (\s -> (x, s))
3
4   -- (<*>) :: ST (a -> b) -> ST a -> ST b
5   stf <*> stx = S (\s ->
6     let (f, s') = app stf s in
7     let (x, s'') = app stx s' in
8     (f x, s''))

```

Flusso di stati e dati

Per comprendere come variano le operazioni, possiamo visualizzare il trasformatore di stato come una scatola nera con un ingresso (lo stato iniziale s) e due uscite (il valore prodotto x e lo stato aggiornato s').

- **Pure:** il valore x viene semplicemente “iniettato” nel sistema. Lo stato s entra ed esce esattamente uguale, senza essere toccato - non c'è alcuna trasformazione, solo un trasporto di informazioni.
- **Funtore (fmap):** lo stato s entra nella scatola st e produce un valore intermedio x e uno stato s' . La funzione g viene applicata solo al valore x , ottenendo $g(x)$. Lo stato s' non viene alterato dalla funzione.
- **Applicativo (<*>):** rappresenta il sequenziamento di due trasformazioni indipendenti;
 - (1) lo stato iniziale s entra nel primo blocco, che produce una funzione f e uno stato intermedio s'
 - (2) questo stato s' viene passato come input al secondo blocco, che produce un valore x e lo stato finale s''
 - (3) solo alla fine la funzione f e il valore x vengono combinati

Qui lo stato fluisce “in serie” tra le due scatole, ma la natura della seconda operazione non dipende dal risultato della prima.

- **Monade (>=):** come nell'applicativo, lo stato fluisce in serie ($s \rightarrow s'$), ma il valore x prodotto dalla prima scatola viene usato per scegliere o costruire la scatola successiva.

3.10.5 Do notation

La **do notation** è una sintassi alternativa (zucchero sintattico) per scrivere computazioni monadiche in modo leggibile, simulando uno stile imperativo. Ogni blocco `do` viene tradotto dal compilatore in una serie di applicazioni degli operatori `>=` e `>>`.

Sia p un'azione (un'espressione di tipo `Monad m => m a`) e `stmts` una sequenza non vuota di comandi. Valgono le seguenti uguaglianze:

- **azione singola:** un blocco `do` contenente una sola azione coincide con l'azione stessa.

```
1 do {p} = p
```

- **sequenziamento:** se un'azione è seguita da altri comandi, viene usato l'operatore `>>`; il risultato della prima azione viene scartato.

```
1 do {p; stmts} = p >> do {stmts}
```

- **binding:** il comando `x <- p` estrae il valore dal contesto monadico di p e lo rende disponibile per i comandi successivi tramite una funzione lambda.

```
1 do {x <- p; stmts} = p >>= \x -> do {stmts}
```

Nota bene: un blocco `do` non può terminare con un comando di estrazione (`x <- p`), ma deve obbligatoriamente terminare con un'azione pura o monadica. Ad esempio, `do x <- getChar` non è un'espressione valida.

3.10.5.1 Esempio Pratico: Tree Relabeling

Per comprendere meglio il funzionamento pratico della monade `ST`, analizziamo il problema del Relabeling di un albero binario. L'obiettivo è prendere un albero ed etichettare ogni sua foglia progressivamente con un numero intero.

Definiamo il tipo di dato per l'albero: i dati risiedono nelle foglie (`F`), mentre i nodi di ramificazione (`R`) servono solo a biforcare la struttura:

```
1 data Tree a = F a | R (Tree a) (Tree a)
```

Per generare gli indici progressivi, definiamo un'azione nello stato chiamata `fresh`, il cui scopo è leggere il contatore corrente e incrementarlo:

```
1 fresh :: ST Int
2 fresh = S (\n -> (n, n + 1))
```

Come funziona ST?

Ricordando che il tipo `ST a` racchiude una funzione di transizione `State -> (a, State)`, quando viene invocata l'azione `fresh` accade questo:

- la funzione accetta lo stato corrente n (l'intero che fa da contatore);
- restituisce la coppia (`valore_prodotto`, `nuovo_stato`)
(che nello specifico è $(n, n + 1)$)
- il valore prodotto n rappresenta il contatore prima dell'incremento, che verrà estratto ed

assegnato alla foglia corrente;

- il nuovo stato $(n + 1)$ non viene memorizzato nella variabile estratta, ma viene preso da Haskell per essere passato implicitamente alla riga di codice successiva;

Sfruttando la `do` notation, l'algoritmo di rietichettatura si scrive in modo sequenziale, mimando la sintassi di un linguaggio imperativo:

```
1 mlabel :: Tree a -> ST (Tree Int)
2 mlabel (F _) = do
3   n <- fresh
4   return (F n)
5
6 mlabel (R l r) = do
7   l' <- mlabel l
8   r' <- mlabel r
9   return (R l' r')
```

Do notation

Il compilatore Haskell converte il blocco `do` rimuovendo lo zucchero sintattico e traducendolo nell'operatore `>=` :

```
1 mlabel (F _) = fresh >= \n -> return (F n)
```

Se sostituiamo l'operatore `>=` con la sua definizione formale implementata per la monade `ST`, l'espressione diventa:

```
1 mlabel (F _) = S (\s ->
2   let (n, s') = app fresh s
3   in app (return (F n)) s')
```

Quindi la computazione procede così:

- (1) la computazione `fresh` viene spaccettata tramite `app` e applicata allo stato iniziale `s`; produce il valore di contatore corrente `n` (pari a `s`) e lo stato incrementato `s'` (pari a `s + 1`).
- (2) l'operatore `>=` cattura questa tupla, estrae il valore `n` e lo passa come argomento alla funzione lambda `\n -> ...`.
- (3) all'interno della lambda viene chiamato `return (F n)`; questa funzione prende la foglia `F n` e la inserisce in un nuovo costruttore `S` che non tocca lo stato, applicando l'azione sul residuo dello stato `s'`;
- (4) lo stato globale aggiornato `s'` fluisce così verso l'esterno, pronto per essere ereditato e ulteriormente modificato dalle successive chiamate ricorsive sui rami dell'albero;

3.10.6 Monad Laws

Perché una struttura sia una monade valida, le operazioni `return` e `>>=` devono soddisfare tre leggi fondamentali:

Identità Destra

Applicare `return` dopo una computazione non deve alterarne il risultato. “Impacchettare” un valore appena estratto è un’operazione neutra.

```
1  -- notazione standard
2  p >>= return = p
3
4  -- do-notation
5  do { x <- p; return x } = do { p }
```

Identità Sinistra

Se inseriamo un valore in un contesto monadico con `return` e lo passiamo a una funzione `f` tramite `bind`, il risultato deve essere identico all’applicazione diretta di `f` al valore.

```
1  -- notazione standard
2  return e >>= f = f e
3
4  -- do-notation
5  do { x <- return e; f x } = do { f e }
```

Associatività

L’ordine con cui concateniamo le operazioni non influenza il risultato finale.

Questa legge permette alla `do`-notation di comportarsi come una sequenza piatta di comandi.

```
1  -- notazione standard
2  ((p >>= f) >>= g) = p >>= (\x -> (f x >>= g))
3
4  -- do-notation
5  do { y <- do { x <- p; f x }; g y } = do { x <- p; y <- f x; g y }
```

3.11 Input/Output

I programmi interattivi sono un problema nel mondo funzionale, perché il valore di una funzione pura dipende solo dai suoi argomenti, mentre input e output sono effettivamente solo dei **side-effects** (per esempio, se una funzione legge un carattere da tastiera, chiamandola due volte consecutive potremmo ottenere due caratteri diversi anche se non le abbiamo passato alcun argomento diverso).

Per risolvere questo paradosso senza rinunciare alla purezza, Haskell adotta l’idea per cui esiste un argomento implicito che rappresenta lo *stato del mondo*.

Si utilizza quindi una Monade e le espressioni del tipo IO sono dette **actions**:

```

1  -- I/O vista come una funzione che 'trasforma il mondo'
2  type IO = World -> World
3
4  -- in realtà, un'azione IO restituisce anche un valore:
5  type IO = World -> (a, World)

```

La definizione del tipo IO è nascosta al programmatore, in modo da impedirgli di manipolare direttamente `World`.

3.11.1 Azioni base

`IO Char` rappresenta quindi il tipo delle azioni che ritornano un carattere, e `IO ()` quelle che ritornano la tupla vuota (simile a `void`), cioè le azioni che sono solo side-effects.

Le primitive di base sono:

```

1  -- per leggere un carattere da tastiera
2  getChar :: IO Char
3
4  -- per scrivere un carattere da tastiera
5  putChar :: Char -> IO ()
6
7  -- per fornire un valore alle azioni
8  -- (utile per esempio nei casi base della ricorsione)
9  return :: a -> IO a

```

Se vogliamo sequenzializzare azioni di IO, utilizziamo gli operatori della monade:

- se il valore restituito da un'azione non ci interessa (es. `putChar`), usiamo `>>`
- se il valore ci interessa, usiamo `>>=` o la `do` notation

Esempi: scrivere una stringa a video e leggere una riga

```

1  putStrLn :: String -> IO ()
2  putStrLn xs = foldr (>>) done (map putChar xs) >> putChar '\n'

```

- `map putChar xs` trasforma una stringa in una lista di azioni `IO ()`
- `foldr (>>)` le concatena in un'unica azione sequenziale, che termina con la stampa del carattere `\n`
- `done` corrisponde a `return ()`

Sequenziare `getChar` è più complicato, perché ci interessa il valore ritornato (il carattere letto).

```

1  getLine = getChar >>= \x ->
2             if x == '\n' then return []
3             else getLine >>= \xs ->
4                 return (x:xs)

```


- `getChar` legge un carattere, che viene passato (sequenzialmente da `>=`) alla funzione lambda che controlla se il carattere letto è il newline - se sì, termina la lettura; altrimenti, richiama la funzione ricorsivamente e ritorna il primo carattere letto concatenato al risultato della chiamata
- `getChar` legge un carattere; il suo viene passato tramite `>=` come argomento `x` alla prima funzione lambda;
- se `x` corrisponde al carattere di newline, la lettura termina e viene restituita una lista vuota `[]` inserita nel contesto monadico tramite `return`;
- altrimenti, `getLine` viene chiamata ricorsivamente per leggere i caratteri rimanenti. Il secondo operatore `>=` estrae la stringa parziale risultante da questa chiamata, legandola alla variabile `xs`, e infine `return (x:xs)` ricompone sequenzialmente la stringa completa posizionando il carattere iniziale in testa alla lista;

3.11.2 Strictness dell'I/O

Anche con le operazioni IO si può usare la `do` notation.

```
1 do
2   v1 <- a1
3   v2 <- a2
4   return (f v1 v2)
```

La funzione `return` è “a senso unico”: permette di inserire un valore puro in un contesto IO, ma non esiste una funzione simmetrica `runIO :: IO a -> a` che permetta di estrarre il valore puro dalla monade.

Se esistesse, si potrebbe scrivere codice come:

```
1 minus :: Int
2 minus = x - y
3 where
4   x = runIO readInt
5   y = runIO readInt
```

Questo programma sarebbe problematico perché violerebbe il **Teorema di Church-Rosser** (o di confluenza) che vale per i linguaggi funzionali puri. Il teorema garantisce che l'ordine con cui il compilatore decide di valutare le sotto-espressioni non influenzi mai il risultato finale.

Se dobbiamo calcolare `minus x - y`, in una funzione pura il compilatore può calcolare `x` e `y` nell'ordine in cui preferisce (il risultato sarà identico).

Se invece esistesse `runIO`, `readInt` chiederebbe all'utente di digitare due numeri. Il compilatore potrebbe decidere di valutare prima `x`, e si otterrebbe `5 - 3`, oppure prima `y`, ottenendo `3 - 5`. Lo stesso programma produrrebbe risultati diversi solo a causa dell'ordine di ottimizzazione interna del compilatore. Poiché l'I/O è *effectful*, l'ordine di lettura è fondamentale.

Per questo, la valutazione nella monade IO è **strict**.

Per esempio,

```
1 do { undefined; return 3 }
```

valuta a `undefined` e non a `3`, anche se `return 3` non avrebbe bisogno dell'eventuale valore calcolato dalla funzione precedente.

3.12 State Thread Monad

Gli algoritmi funzionali puri sulle liste sono eleganti, ma non sempre efficienti. Sebbene molti mantengano la stessa complessità asintotica temporale dei programmi iterativi, la ricorsione introduce un overhead di spazio dovuto ai record di attivazione nello stack. Inoltre, alcune strutture dati sono impossibili da realizzare in modo efficiente senza memoria mutabile: per esempio, le hash table e i dizionari richiedono accesso diretto e modifica in-place degli array.

Per risolvere questo problema, il modulo `Control.Monad.ST` fornisce una monade che permette di modificare la memoria in maniera simile ad un linguaggio imperativo.

La struttura di `Control.Monad.ST` è analoga a quella della monade State Transformer. Tuttavia, gli elementi della variabile di tipo `s` non sono accessibili (come nel caso di `World`), se non attraverso le operazioni definite.

```
1 type ST s a = s -> (a, s)
```

- `s` rappresenta quindi una specie di etichetta che identifica un thread di esecuzione (e non si possono trasmettere valori da un thread ad un altro)

All'interno di un thread `s` si possono creare delle vere e proprie variabili mutabili, chiamate `STRef s a` (reference a un valore di tipo `a` dentro il thread `s`).

Le principali primitive esposte da Haskell per lavorare su una variabile `STRef` sono:

- `newSTRef :: a -> ST s (STRef s a)`
crea una nuova variabile mutabile, la inizializza con un valore puro e restituisce il suo riferimento
- `readSTRef :: STRef s a -> ST s a`
legge il valore memorizzato nella variabile
- `writeSTRef :: STRef s a -> a -> ST s ()`
sovrascrive il contenuto della variabile (restituisce `()` perché serve solo il suo side-effect)
- `modifySTRef :: STRef s a -> (a -> a) -> ST s ()`
permette di aggiornare il valore di una variabile applicandovi una funzione (esiste in versione lazy - `modifySTRef` -, e strict - `modifySTRef'`)

Esempio: Fibonacci

Tramite tupling si può ottenere un Fibonacci lineare, ma che, essendo ricorsivo, occupa $\Theta(n)$ in termini di spazio.

Tramite la monade `STRef`, si può ottenere una complessità spaziale di $\Theta(1)$

```

1  -- repeatFor simula un ciclo
2  repeatFor n = foldr (>>) done . replicate n
3
4  fibST :: Int -> ST s Integer
5  fibST n = do
6    a <- newSTRef 0
7    b <- newSTRef 1
8    repeatFor n (do
9      x <- readSTRef a
10     y <- readSTRef b
11     writeSTRef a y
12     writeSTRef b $(x + y))
13    readSTRef a

```

- `a` e `b` vengono allocate ed inizializzate a `0` e `1`
- il blocco interno a `repeatFor` viene eseguito n volte; ad ogni iterazione:
 - si leggono i valori di `a` e `b`
 - si aggiorna il valore di `a` con `y` e quello di `b` con la somma (calcolata in modo strict con `$!`, che forza la valutazione immediata) `$! (x + y)`
 - alla fine del ciclo, si legge e restituisce il valore accumulato in `a`

Il valore ritornato è però di tipo `ST s Integer`. Bisogna quindi estrarre un intero puro.

Serve quindi una funzione `runST` analoga a `runState` per gli State Transformer.

Questa funzione esiste, ma ha un tipo che non appartiene ai tipi “standard” di Haskell (à la Hindley Milner) - contiene infatti un $\forall$ all’interno del tipo invece che all’inizio (apparterrebbe quindi ai tipi del Lambda Calcolo Polimorfo / F2 (vedi [appunti del corso “Linguaggi di programmazione”](#))):

```

1  runST :: (forall s. ST s a) -> a

```

Un tipo di questo genere si dice **rank 2** (mentre i tipi di Haskell sono rank 1).

Lo stato deve essere inaccessibile (poiché il $\forall$ è interno, la variabile è “locale”: esiste solo all’interno di quella computazione e non può far parte del tipo di ritorno `a`) e quindi espressioni come

```

1  let v = runST (newSTRef True) in runST (readSTRef v)

```

non ben tipate. Qui il tipo di ritorno sarebbe `STRef s Bool`, tipo che contiene `s`, ma il type checker non può unificare `STRef s a` con un tipo che dipende da `s`.

Questo controllo viene implementato da Haskell per evitare i bug di mutabilità (race conditions, problemi di puntatori ecc) che non dovrebbero appartenere ai linguaggi funzionali. (Se permettessimo a contesti diversi di scambiarsi riferimenti di memoria interni, un thread potrebbe modificare la memoria usata da un altro.)

Per estrarre un valore con `runST`, basta quindi scrivere qualcosa come `fib n = runST (fibST n)`.

3.12.0.1 Array mutabili

Un array in Haskell ha tipo `STArray s i e`, dove `s` è il thread a cui appartiene, `i` è il tipo dell'indice, ed `e` il tipo dei valori contenuti.

Il tipo `i` deve essere istanza della classe `Ix` (per *index*) che contiene tipi enumerabili (con operazioni di `succ`) come `Int`, `Char` ecc.

I metodi fondamentali su `STArray` sono:

- `newListArray (min, max) lista` - inizializza un array mutabile specificando l'intervallo degli indici e inserendo gli elementi di una lista iniziale
- `newArray (min, max) valoreIniziale` - inizializza un array mutabile vuoto o riempito di un valore di default
- `readArray xa indice` - legge e restituisce il valore all'indice specificato
- `writeArray xa indice valore` - modifica il contenuto di `xa` sovrascrivendo il valore all'indice dato con `valore` (non ritorna nulla)
- `getBounds xa` - restituisce la tupla `(min, max)` degli indici dell'array
- `getElems xa` - prende gli elementi dall'array e li copia in una lista haskell
- `runSTArray` - come per `runST`, esegue un blocco di codice `ST` che crea e modifica un array mutabile, e permette di ottenere l'Array "puro"

(Il tipo `STArray` si trova nel modulo `Data.Array.ST`, mentre i metodi si trovano in `Data.Array.MArray`)

3.13 Kleisli composition (composizione monadica)

Nel mondo della programmazione funzionale standard, se abbiamo due funzioni pure `f :: a -> b` e `g :: b -> c`, possiamo comporle facilmente utilizzando `(.)`.

Quando introduciamo i side-effects tramite monadi, le funzioni restituiscono valori impacchettati in contesti monadici (assumendo la forma delle cosiddette **frecche di Kleisli**, e.g. `a -> m b`). L'operatore standard `(.)` non può comporre questo tipo di funzioni.

Per consentire la composizione di funzioni monadiche, Haskell fornisce l'operatore della **composizione di Kleisli**:

```
1 (>=>) :: Monad m => (a -> m b) -> (b -> m c) -> (a -> m c)
```

L'operatore viene implementato come

```
1 (f >=> g) x = f x >=> g
```

(viene calcolato il risultato di `f x` (che produce un `m b`), e questo viene passato alla funzione `g` tramite `>=>`).

L'ordine di esecuzione è da sinistra verso destra (prima `f` e poi `g`), opposto rispetto alla composizione classica.

Esiste anche la versione speculare dell'operatore - `<=<` -, definita invertendo l'ordine dei fattori:

```
1 (g <=< f) x = f x >>= g
```

Definizione di `>>=` in termini di `>=>`

Si può anche definire `>>=` in termini di `>=>`.

Osserviamo i tipi dei due operatori:

```
1 (>=>) :: Monad m => (a -> m b) -> (b -> m c) -> (a -> m c)
2 (>>=) :: Monad m => m a -> (a -> m b) -> m b
```

L'operatore `>=>` lavora in modo molto generale con tre parametri di tipo (`a`, `b` e `c`). Possiamo quindi istanziare `a` con un tipo meno generale `m b`. Il tipo di `>=>` si specializza quindi in questo modo:

```
1 (>=>) :: Monad m => (m b -> m b) -> (b -> m c) -> (m b -> m c)
```

Osservando questa firma, notiamo che il primo argomento è una funzione `mb -> mb`. La funzione più semplice che ha questo tipo è l'identità.

Se passiamo quindi l'identità a `>=>`, otteniamo una funzione parzialmente applicata `id >=>`, che accetta una funzione `f :: b -> mc` e restituirà una nuova funzione `mb -> mc`.

Se quindi abbiamo un valore monadico `p :: mb` e una funzione `f :: b -> mc`, possiamo scrivere:

```
1 (p >>= f) = (id >=> f) p
```

Per evitare di dichiarare `p` e `f`, notiamo che in `(id >=> f) p` la funzione viene fornita prima del valore, mentre nel bind classico l'ordine è invertito (`p >>= f`).

Possiamo quindi scrivere più elegantemente:

```
1 (>>=) = flip (id >=> )
```

Esprimendo `>>=` in termini di `>=>`, le Monad Laws diventano più "eleganti".

Def. 13: Monoide

In matematica, un **monoide** è una struttura algebrica elementare composta da:

- un insieme di elementi
- un'operazione binaria (e.g. $*$)
- un elemento neutro (e)

che rispetta due proprietà fondamentali:

- associatività $((a * b) * c = a * (b * c))$
- identità (destra e sinistra) $(a * e = e * a = a)$

(un monoide è un semigrupp con elemento neutro)

(quindi si ha che $\text{gruppi} \subseteq \text{monoidi} \subseteq \text{semigruppi}$)

Nel mondo della programmazione funzionale senza monadi, le funzioni pure e a loro composizione formano un monoide (con operazione $(.)$ ed elemento neutro id).

Se scriviamo le proprietà di questo monoide per tre generiche funzioni, otteniamo:

- $\text{id} . f = f$ - identità sinistra
- $f . \text{id} = f$ - identità destra
- $(f . g) . h = f . (g . h)$ - associatività

Se invece passiamo al mondo monadico, per esprimere le Monad Laws tramite $\gg=$, siamo costretti, come abbiamo visto, a introdurre notazioni non uniformi:

- $\text{return } e \gg= f = f \ e$
- $p \gg= \text{return} = p$
- $((p \gg= f) \gg= g) = p \gg= (\lambda x \rightarrow (f \ x \gg= g))$

L'operatore di Kleisli permette invece di ridefinire le Monad Laws in maniera completamente simmetrica a quelle dei monoidi:

- $\text{return} \gg= f = f$
- $f \gg= \text{return} = f$
- $(f \gg= g) \gg= h = f \gg= (g \gg= h)$

3.14 Functor e Applicative come Monad

Fino ad ora abbiamo visto le classi di tipi presentate in ordine crescente di complessità (da `Functor` a `Monad`). È tuttavia possibile compiere il percorso opposto: se un tipo istanzia la classe più potente (`Monad`), le operazioni delle classi superiori possono essere ridefinite in modo del tutto naturale.

3.14.1 Definire `fmap` e `<*>`

L'operatore `fmap` dei funtori può essere espresso interamente a partire dalle primitive di un `Applicative` sfruttando `pure` e `<*>`:

```
1 fmap f x = pure f <*> x = f <$> x
```

Analogamente, l'operatore applicativo `<*>` può essere ricostruito sfruttando il bind (`>=>`) e la funzione `return`:

```
1 f <*> x = f >=> \g -> x >=> \y -> return (g y)
```

Questo meccanismo estrae sequenzialmente la funzione pura `g` e il valore puro `y` dai rispettivi contesti monadici, applica la funzione e ne impacchetta nuovamente il risultato. Ciò dimostra formalmente che **Monad** è un sottotipo di **Applicative**, il quale a sua volta è un sottotipo di **Functor**.

3.15 Funzione `join`

Quando si lavora con le monadi, capita spesso di ottenere contesti duplicati o stratificati (e.g. lista di liste o `Maybe (Maybe a)`). La funzione `join` permette di appiattire un'applicazione annidata di un tipo monadico:

```
1 join :: Monad m => m (m a) -> m a
2 join mmx = do
3   mx <- mmx
4   x <- mx
5   return x
```

- `mx <- mmx` estrae la monade interna `mx` rimuovendo il primo stato monadico
- `x <- mx` estrae il valore puro dalla monade interna
- `return x` prende il valore puro e lo inserisce nuovamente nel contesto

oppure semplicemente:

```
1 join :: Monad m => m (m a) -> m a
2 join mmx = do
3   mx <- mmx
4   mx
```

(infatti si ha che `do { x <- mx; return x }` è equivalente a `mx` (prima Monad Law, p.62))

3.15.1 Definizione di bind tramite `join`

Nella Teoria delle Categorie, una monade non è definita tramite bind, ma è descritta come un funtore equipaggiato con due trasformazioni: un'unità (`return`) e una moltiplicazione (`join`).

In Haskell, possiamo ricavare il legame tra i due approcci scrivendo il bind in questo modo:

```
1 m >=> f = join (fmap f m)
```

Essenzialmente, fare un bind significa applicare una funzione che genera una struttura annidata (`fmap f m` produce `m (m b)`), e poi usare un `join` per "appiattire" il risultato.

Anche le Monad Laws possono quindi essere ridefinite con `join` e `fmap`:

```
1 join . return = id
```

- se inseriamo un elemento in un contesto tramite `return` e poi uniamo i contesti con `join`, l'effetto si annulla

```
1 join . fmap return = id
```

- se prendiamo una monade, mappiamo `return` al suo interno (creando una struttura sdoppiata) e poi applichiamo `join`, torniamo alla monade di partenza

```
1 join . join = join . fmap join
```

- se abbiamo tre livelli di monade annidati `(m (m (m a)))`, non importa se decidiamo di “schiacciare” prima i due livelli più interni o i due livelli più esterni: il risultato finale sarà identico

3.16 Generic Programming

Le classi in Haskell permettono forme di programmazione generica basata sul fatto che:

- gli operatori hanno lo stesso nome (overloading) (modificano il loro comportamento in base all'istanza specifica)
- ogni classe di tipi rispetta leggi matematiche precisi (es. Monad Laws); in questo modo, il comportamento del codice generico è prevedibile e corretto

Il generic programming in Haskell fonde il polimorfismo parametrico (funzioni che lavorano con tipi generici) con il polimorfismo ad-hoc (fornito dalle type classes). Si può scrivere un algoritmo una sola volta ed applicarlo a qualsiasi tipo che implementi l'interfaccia richiesta.

3.16.1 mapM e filterM

Un esempio è la funzione `mapM`, una generalizzazione monadica di `map` (`map` applica una funzione pura `a -> b` agli elementi di una lista, mentre `mapM` applica una funzione che genera un tipo monadico `a -> m b`).

```
1 mapM :: Monad m => (a -> m b) -> [a] -> m [b]
2 mapM f [] = return [] -- "impacchettamento"
3 mapM f (x:xs) = do y <- f x
4                  ys <- mapM f xs
5                  return (y:ys)
```

Questa funzione consente quindi di gestire facilmente side-effects nella computazione.

In maniera analoga, la funzione `filterM` generalizza `filter`.

`filter` prende un predicato puro `a -> Bool` e una lista `[a]`, e restituisce solo gli elementi che soddisfano il predicato. La controparte monadica `filterM` accetta un predicato che genera side-effect (monadici):


```

1 filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
2 filterM p [] = return []
3 filterM p (x:xs) = do b <- p x
4                      ys <- filterM p xs
5                      return (if b then x:ys else ys)

```

Powerset tramite non-determinismo

Si possono sfruttare `filterM` e il non-determinismo delle liste di Haskell per fornire un'implementazione molto elegante della generazione di powerset:

```

1 filterM (\x -> [True, False]) [1, 2, 3]
2 -- si ottiene [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]

```

Infatti, quando si usa la monade lista, ogni operazione di bind effettua un prodotto cartesiano di tutte le possibilità. Il predicato `\x -> [True, False]` dice essenzialmente, dato un `x`, di considerare in modo non-deterministico sia il caso in cui il predicato sia `True` che quello in cui sia `False`.

Quindi:

- per il numero `1`, la monade si “sdoppia”: in un ramo di esecuzione, `1` viene tenuto, e nell'altro viene scartato
- per `2`, ciascuno dei rami precedenti si sdoppia nuovamente
- e la stessa cosa avviene per `3`

Alla fine della computazione la monade avrà esplorato l'intero albero delle decisioni combinatorie, e restituirà l'unione di tutti i cammini possibili (ovvero tutte le combinazioni di elementi, il powerset!).

3.17 Monoidi, Foldable e Traversable

3.17.1 Monoid

Un **monoid** è una struttura algebrica dotata di un'operazione associativa e un'identità (vedi p.69).

In Haskell, la classe `Monoid` è definita in questo modo:

```

1 class Monoid a where
2     mempty  :: a           -- identità
3     mappend :: a -> a -> a -- scritto anche <>
4     mconcat :: [a] -> a
5     mconcat = foldr mappend mempty

```

Le operazioni devono aderire a queste leggi:

- `mempty <> x = x`
- `x <> mempty = x`
- `x <> (y <> z) = (x <> y) <> z`

L'esempio classico di monoido è quello delle liste con `[]` e `++`.

```
1 instance Monoid [a] where
2     mempty = []
3     mappend = (++)
```

Anche `Maybe` si può definire come monoido:

```
1 instance Monoid a => Monoid (Maybe a) where
2     mempty = Nothing
3     Noting 'mappend' my = my
4     mx 'mappend' Nothing = mx
5     (Just x) 'mappend' (Just y) = Just(x 'mappend' y)
```

- notiamo che questo approccio si comporta nel modo opposto rispetto a `mapM`! qui, si propagano i `Just`, non i `Maybe` (si “ripuliscono” le strutture dati)

3.17.1.1 Wrapper Types

Alcuni tipi possono essere **monoidi rispetto a più operazioni** (e.g. gli interi sono monoidi per somma e prodotto).

Dato che Haskell vieta di definire due istanze diverse della stessa classe per lo stesso tipo, si usano i **wrapper types**, definiti tramite `newtype`.

Per esempio, per i booleani, si possono definire i due monoidi `All` per l'operazione di AND e `Any` per l'operazione di OR.

```
1 newtype All = All Bool
2 getAll (All b) = b
3
4 instance Monoid All where
5     mempty = All True
6     (All b) 'mappend' (All c) = All (b && c)
7
8 newtype Any = Any Bool
9 getAll (Any b) = b
10
11 instance Monoid Any where
12     mempty = Any True
13     (Any b) 'mappend' (Any c) = Any (b || c)
```

3.17.2 Foldable

La classe `Foldable` raccoglie tutte le strutture dati i cui elementi possono essere “ridotti” o accumulati fino ad ottenere un singolo valore. Se gli elementi all'interno della struttura sono dei monoidi, l'accumulazione viene fatta in modo “automatico” tramite `mempty` e `mappend`.

La sua interfaccia è:

```
1 class Foldable t where
2     fold    :: Monoid a => t a -> a
3     foldMap :: Monoid b => (a -> b) -> t a -> b
```

```

4 foldr :: (a -> b -> b) -> b -> t a -> b
5 foldl :: (b -> a -> b) -> b -> t a -> b

```

- `fold` prende una struttura di monoidi e li “schiaccia”

```

1 -- esempio: liste
2 fold [] = mempty
3 fold (x:xs) = x 'mappend' fold xs

```

- `foldMap` prende una funzione trasformatrice che mappa gli elementi della struttura in un monoide, e poi effettua il `fold`
- `foldr` e `foldl` non necessitano che `a` o `b` sia un monoide, perché viene fornita direttamente la funzione di composizione

Molte delle funzioni dei foldable sono interdefinibili (per esempio, `fold` si può definire come `foldMap id`). Quindi, quando si implementa un'istanza di `Foldable`, basta implementare una delle due tra `foldr` e `foldMap`, e il compilatore deriverà tutte le altre funzioni della classe.

Inoltre, tutti i foldable sono riducibili ad una lista tramite

```

1 toList = foldMap (:[])

```

3.17.3 Traversable

`Traversable` è un'evoluzione del concetto di `Functor` (mappa una funzione su una struttura) - e una generalizzazione di `map` - che permette di gestire computazioni con side-effects.

```

1 class (Functor t, Foldable t) => Traversable t where
2   traverse :: Applicative f => (a -> f b) -> t a -> f (t b)

```

- la funzione `traverse` attraversa la struttura `t a`, applica ad ogni elemento la funzione (con side-effects) `a -> f b`, ed estrae l'effetto accumulato “portandolo fuori” dalla struttura (da `a -> f b` e `t a` si passa a `f (t b)`)

```

1 traverse g [] = pure []
2 traverse g (x:xs) = pure (:) <*> g x <*> traverse g xs
3
4 -- esempio:
5 pred n = if n > 0 then Just (n-1) else Nothing
6
7 > traverse pred [1,2,3]
8 Just [0,1,2]
9 -- ^ vediamo che ‘porta fuori’ l'effetto
10 -- (una map darebbe [Just 0, Just 1, Just 2])
11 > traverse pred [2,1,0]
12 Nothing -- il Nothing causato da 0 si propaga

```

Alberi come traversable

```

1 instance Traversable Tree where
2   traverse g (Leaf x) = pure Leaf <*> g x
3   traverse g (Node l r) = pure Node <*> traverse g l <*> traverse g r

```

`pure Leaf <*> g x`

- `Leaf` ha tipo `b -> Tree b`
- `g` ha tipo `a -> f b`
- `pure Leaf` inserisce il costruttore `Leaf` nel contesto - ora ha tipo `f (b -> Tree b)`
- `<*> g x` - si usa `<*>` per passare il valore dentro `g x` (di tipo `f b`) alla funzione dentro `pure Leaf`
- in questo modo, si ottiene una foglia nel contesto: `f (Tree b)`

`pure Node <*> traverse g l <*> traverse g r`

- le due chiamate `traverse g` restituiscono i sottoalberi modificati e contestualizzati: `f (Tree b)`
- come sopra, `Node` va inserito nel contesto tramite `pure`, e come prima si usa `<*>` per passare i (due questa volta) argomenti al costruttore